ANALISI E PROGETTAZIONE DI ALGORITMI · A.A. 2025/26

Guida completa allo scritto

Tutto il corso riorganizzato attorno ai **4 esercizi della prova scritta**: teoria spiegata in modo semplice, schemi, diagrammi, procedure passo-passo e le due prove d'esame (16/06/2026 e 14/07/2026) svolte per intero.

1

Algoritmi di ordinamento

Pseudocodice (Insertion, Selection, Heap, Quick Sort) e simulazione su un array, iterazione per iterazione.

2

Equazioni di ricorrenza

Andamento asintotico con il **metodo dell'albero di ricorsione**: tabella dei costi, serie, altezze.

3

Da ricorsivo a iterativo

Simulare con precisione una funzione ricorsiva: **call**, **last**, **ret** e uno stack per ogni variabile.

4

Programmazione dinamica

Sottoproblemi, ricorrenza, algoritmo iterativo con costi e simulazione su un input (tabelle).

Fonti: appunti delle lezioni (marzo–maggio 2026), dispensa ricostruita dagli appunti, dispense dell'altro corso (solo per integrare), tracce d'esame.

Indice

0 Com'è fatto lo scritto 4

0.1 La struttura della prova 4

0.2 I 4 esercizi: cosa chiedono e dove studiarli 4

0.3 Le due prove del 2026 a colpo d'occhio 5

0.4 Come usare questa guida 5

1 Fondamenti: misurare un algoritmo 6

1.1 Tempo di esecuzione e dimensione dell'input 6

1.2 Le tre notazioni: O, Ω, Θ 6

1.3 Il kit di logaritmi e serie (serve per l'Esercizio 2) 7

1.4 Un primo esempio completo: la sottosequenza di somma massima 7

2 Algoritmi di ordinamento 8

2.1 Panoramica e "contare le iterazioni" 8

2.2 Insertion Sort 9

2.3 Selection Sort 10

2.4 Merge Sort (divide et impera) 11

2.5 Heap e Heap Sort 12

2.6 Quick Sort e Partiziona 16

2.7 Limite inferiore: nessuno fa meglio di nlogn 18

3 Equazioni di ricorrenza con l'albero di ricorsione 19

3.1 L'idea: il tempo è la somma dei nodi dell'albero 19

3.2 Il metodo in 5 passi 19

3.3 La ragione r decide tutto 20

3.4 Caso 1 — un ramo, molteplicità costante 21

3.5 Caso 2 — un ramo, molteplicità non costante 21

3.6 Caso 3 — due rami 23

4 Da algoritmo ricorsivo ad algoritmo iterativo 25

4.1 Cosa fa davvero una chiamata ricorsiva 25

4.2 Gli ingredienti dello schema 25

4.3 Le tre regole 26

4.4 Lo schema generale 27

4.5 Esempio guidato: il massimo di un array (dagli appunti) 28

4.6 Esame 16/06/2026 — Algoritmo(T, i) 29

4.7 Esame 14/07/2026 — Algoritmo(T, a, b) 31

5 Programmazione dinamica 33

5.1 Perché serve: i sottoproblemi che si ripetono 33

5.2 Il metodo della PD (e lo schema di risposta) 33

5.3 Taglio del tronco 35

5.4 Schedulazione di intervalli pesati 35

5.5 Sottosequenza comune più lunga (LCS) 37

5.6 Segmentazione di un testo integrazione 37
5.7 Albero binario di ricerca ottimo 38
5.8 Zaino 0/1 40
5.9 Cammini minimi da sorgente singola (Bellman-Ford come PD) 41
5.10 Cammini minimi tra tutte le coppie 44
5.11 Cammino semplice massimo: quando la PD non basta 45
5.12 Riepilogo della programmazione dinamica 45

6 Algoritmi greedy 46
6.1 L'idea 46
6.2 Selezione di intervalli (non pesati) 46
6.3 Schedulazione con tempo medio di completamento minimo integrazione 47
6.4 Zaino frazionario integrazione 47

7 Code di priorità e Dijkstra 48
7.1 Rilassamento e idea di Dijkstra 48
7.2 Coda di priorità con un min-heap 48
7.3 Dijkstra e la sua complessità 49

8 Formulario e checklist finali 50
8.1 Tutte le complessità 50
8.2 Formule da avere in mano 50
8.3 Le quattro checklist 50

PRIMA DI TUTTO

0 Com'è fatto lo scritto

Lo scritto è sempre lo stesso "modulo" da 4 esercizi. Se sai cosa ti chiedono e con quale metodo devi rispondere, metà del lavoro è fatto.

0.1 La struttura della prova

- **Tempo:** 2 ore e 30 minuti.
- **4 esercizi** con lo **stesso peso**. Secondo quanto riportato a lezione, per passare servono circa **2 esercizi e mezzo** svolti correttamente.
- Nell'esercizio 1 il testo dice: *"In assenza di una corretta motivazione, lo svolgimento sarà considerato errato"*. Ogni passaggio va **giustificato in una riga**.
- La programmazione dinamica (o greedy) c'è **sempre**.

REGOLE DEL PROF (DA RICEVIMENTO)

- Ricorrenze: si accetta **solo il metodo dell'albero di ricorsione** (niente Teorema Master o altri metodi).
- Ricorsivo → iterativo: usa lo schema **call / last / ret + uno stack per ogni variabile**. Uno stack "di stato" al posto di **last**, o un unico stack di "frame", sono considerati **non ammessi**.
- Pseudocodice: quello **"discusso a lezione"**, cioè le versioni in questa guida.

0.2 I 4 esercizi: cosa chiedono e dove studiarli

Esercizio	Cosa ti chiedono (esempi reali)	Metodo	Tempo
1 · Ordinamento	(i) pseudocodice di 2 algoritmi (es. **HeapSort + BuildHeap + Heapify** e **SelectionSort**; oppure **QuickSort + Partiziona** e **InsertionSort**); (ii)–(iii) simularli su un array mostrando l'array dopo certe iterazioni.	Esecuzione "a mano" riga per riga · Cap. 2	~30 min
2 · Ricorrenze	Trovare l'andamento asintotico di una $T(n)$, es. $T(n) = 3\sqrt[3]{n^2}\,T(\sqrt[3]{n}) + 7n$ oppure $T(n) = T(\frac{3n}{5}) + 4T(\frac{n}{2}) + n$.	Albero di ricorsione + tabella dei costi · Cap. 3	~30 min
3 · Iterativo	Scrivere un algoritmo iterativo che **simuli precisamente** una funzione ricorsiva su alberi binari (con due chiamate ricorsive).	Schema call/last/ret/stack · Cap. 4	~35 min
4 · PD	Descrivere l'approccio di PD per un problema (es. **albero di ricerca ottimo**, **cammini minimi da sorgente singola**): decomposizione, ricorrenza, algoritmo iterativo + costo; poi **simularlo** su un input.	Schema in 5 punti + tabella · Cap. 5–6	~45 min

Restano circa 10 minuti per rileggere. Comincia dall'esercizio che ti riesce meglio: hanno tutti lo stesso peso.

0.3 Le due prove del 2026 a colpo d'occhio

TEMA DEL 16/06/2026

1. Pseudocodice di **QuickSort + Partiziona** e di **InsertionSort** iterativo; InsertionSort su $A = [7, 6, 8, 5, 1, 9, 8, 0, 2, 4, 3, 1]$ (3ª, 7ª, 10ª iterazione); Partiziona su $A = [3, 2, 8, 5, 1, 9, 2, 0, 2, 7, 3, 1]$ (ogni iterazione + partizioni). → **§2.2, §2.6**
2. $T(n) = T(\frac{3n}{5}) + 4\,T(\frac{n}{2}) + n$. → **§3.7**
3. Iterativo di `Algoritmo(T, i)` con funzioni esterne G, H. → **§4.7**
4. PD per i **cammini minimi da sorgente singola** + simulazione su un grafo. → **§5.9**

TEMA DEL 14/07/2026

1. Pseudocodice di **HeapSort, BuildHeap, Heapify** e **SelectionSort**; BuildHeap su $A = [3, 2, 1, 0, 8, 1, 2, 5, 2, 7, 3, 9]$ (indici 0...11, ogni iterazione); HeapSort (2ª, 5ª, 8ª iterazione). → **§2.3, §2.5**
2. $T(n) = 3\sqrt[3]{n^2}\,T(\sqrt[3]{n}) + 7n$. → **§3.6**
3. Iterativo di `Algoritmo(T, a, b)` (verifica di un BST). → **§4.8**
4. PD per l'**albero binario di ricerca ottimo** con $A = [1..5]$, $F = [\frac{4}{19}, \frac{8}{19}, \frac{2}{19}, \frac{1}{19}, \frac{4}{19}]$. → **§5.7**

0.4 Come usare questa guida

IDEA CHIAVE
Il concetto da capire (e ricordare) prima di tutto il resto.

ATTENZIONE
Errori tipici che costano l'esercizio.

NEL COMPITO SCRIVI COSÌ
Il formato della risposta da riportare sul foglio.

Ogni capitolo si apre con **"cosa ti chiedono"** e **"cosa devi saper fare"**, poi spiega la teoria con esempi, poi svolge per intero l'esercizio delle prove reali. Alla fine trovi un **formulario** con tutte le complessità e le checklist da ripassare prima dell'esame.

CONVENZIONI

- $\log$ senza base indica $\log_2$. Assegnamento: $\leftarrow$. Gli array partono da **1** salvo dove indicato: **il tema del 14/07 usa indici da 0** (vedi §2.5 per la conversione).
- Pseudocodice con indentazione per i blocchi, come negli appunti e nelle tracce.

BASE PER TUTTI GLI ESERCIZI

1 Fondamenti: misurare un algoritmo

Notazione asintotica, logaritmi e serie: gli attrezzi che userai per motivare ogni risposta e, soprattutto, per risolvere le equazioni di ricorrenza.

1.1 Tempo di esecuzione e dimensione dell'input

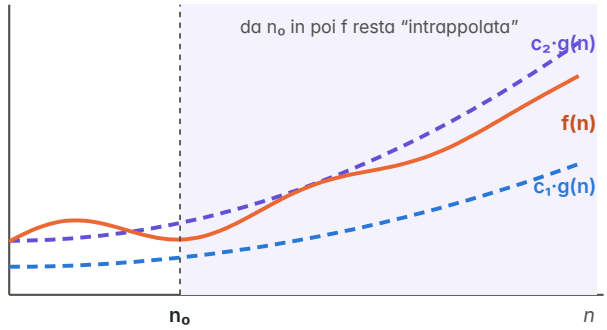
Per un algoritmo A si definisce $T_A(n)$ = **numero di operazioni elementari** (assegnamenti, confronti, somme, accessi ad array) eseguite su un input di dimensione n. Ogni operazione elementare costa una costante, quindi non conta il computer: conta **come cresce** $T(n)$.

La **dimensione dell'input** è la memoria necessaria per rappresentarlo. Per un array di n numeri è n; per un singolo numero K scritto in binario è circa $\log K$ bit. Questo dettaglio torna nello zaino 0/1 (§5.8): un algoritmo $\Theta(nK)$ **non** è polinomiale.

1.2 Le tre notazioni: O, Ω, Θ

Notazione	Significa	Definizione
$f = O(g)$	f cresce **al più** come g ("$\leq$")	$\exists c, n_0 > 0 : \ f(n) \leq c\,g(n)\ \forall n \geq n_0$
$f = \Omega(g)$	f cresce **almeno** come g ("$\geq$")	$\exists c, n_0 > 0 : \ f(n) \geq c\,g(n)\ \forall n \geq n_0$
$f = \Theta(g)$	f cresce **esattamente** come g ("$=$")	$\exists c_1, c_2, n_0 : \ c_1 g(n) \leq f(n) \leq c_2 g(n)$

$f = \Theta(g) \iff f = O(g)$ e $f = \Omega(g)$.



Θ: da n_0 in poi f sta tra $c_1 g$ e $c_2 g$.

IL CRITERIO DEL LIMITE (IL MODO PIÙ VELOCE)

Calcola $L = \lim_{n\to\infty} \frac{f(n)}{g(n)}$:

$L = 0$	$0 < L < \infty$	$L = \infty$
$f = O(g)$, non Θ	$f = \Theta(g)$	$f = \Omega(g)$, non Θ
f cresce meno	stessa crescita	f cresce di più

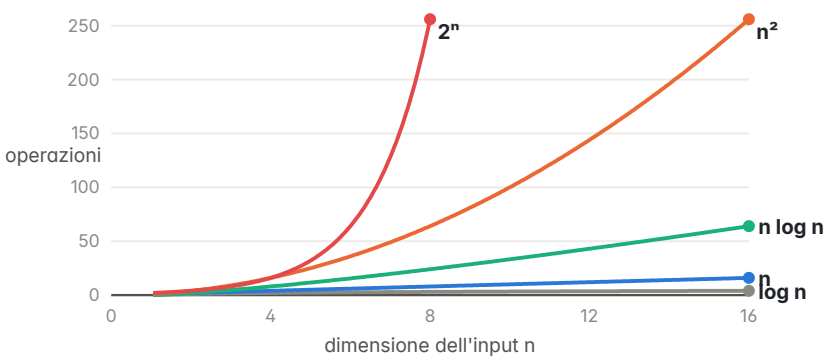
Esempio: $f = 3n^2 + 5n$, $g = n^2$: $\lim \frac{3n^2+5n}{n^2} = 3 \Rightarrow f = \Theta(n^2)$.

La scala della crescita

Dal più lento al più veloce:

$$1 \ll \log\log n \ll \log n \ll \sqrt{n} \ll n \ll n\log n \ll n^2 \ll n^3 \ll 2^n \ll n!$$

Costanti moltiplicative e termini di ordine inferiore **non contano**: $5n^2 + 100n\log n = \Theta(n^2)$. Nemmeno la base del logaritmo ($\log_a n = \frac{\log_b n}{\log_b a}$).



Funzioni di crescita a confronto: con $n = 16$, 2^n è già fuori scala.

ESERCIZIO TIPO: TROVARE LE COSTANTI (DALLE DISPENSE DELL'ALTRO CORSO)

Mostrare che $n\ln(e^2 n) = \Theta(\ln\sqrt{n^n})$. Semplifico: $f(n) = n(\ln e^2 + \ln n) = 2n + n\ln n$; $g(n) = \ln n^{n/2} = \frac{n}{2}\ln n$. Cerco $c_1\frac{n}{2}\ln n \le 2n + n\ln n \le c_2\frac{n}{2}\ln n$; divido per $\frac{n}{2}\ln n$ (con $n > 1$): $c_1 \le \frac{4}{\ln n} + 2 \le c_2$. Il termine centrale è sempre $> 2 \Rightarrow c_1 = 2$; decresce, e per $n \ge n_0 = e$ vale al più $\frac{4}{1} + 2 = 6 \Rightarrow c_2 = 6$. **Vera**, con $c_1 = 2,\ c_2 = 6,\ n_0 = e$ (in pratica $n_0 = 3$).

1.3 Il kit di logaritmi e serie (serve per l'Esercizio 2)

LOGARITMI

- $\log_a(xy) = \log_a x + \log_a y \ \cdot\ \log_a \frac{x}{y} = \log_a x - \log_a y$
- $\log_a x^k = k\log_a x \ \Rightarrow\ \log n^{1/2} = \frac{1}{2}\log n$, $(\log n^{1/3})^2 = \frac{1}{9}\log^2 n$
- Cambio base: $\log_a b = \dfrac{\log_c b}{\log_c a}$
- **Scambio**: $a^{\log_b c} = c^{\log_b a} \ \Rightarrow\ 3^{\log_3 \log_3 n} = \log_3 n$, $7^{\log_3 n} = n^{\log_3 7}$
- $a^{\log_a x} = x \ \cdot\ \log_{1/2} n = -\log_2 n$

SERIE GEOMETRICA $\sum r^i$

- $r \neq 1$: $\displaystyle\sum_{i=0}^{k} r^i = \frac{r^{k+1} - 1}{r - 1}$
- $r < 1$: $\displaystyle\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r}$ (costante!)
- $r = 1$: $\displaystyle\sum_{i=0}^{k} 1 = k + 1$
- Aritmetica: $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$
- Utile: $\displaystyle\sum_{i\ge 0} i\,x^i = \frac{x}{(1-x)^2} \Rightarrow \sum_{i\ge 0}\frac{i}{2^i} = 2$

QUANDO LA DIMENSIONE SCENDE, QUANTI LIVELLI CI SONO?

la dimensione al livello i è...	condizione di caso base	altezza h
$n - i$ (togli 1)	$n - i \le 1$	$\approx n$
n/b^i (dividi per b)	$n/b^i \le 1$	$\log_b n$
$(c/d)^i\, n$, es. $(\frac{3}{5})^i n$	$(\frac{3}{5})^i n \le 1$	$\log_{5/3} n$
n^{1/b^i} (radice b-esima)	$n^{1/b^i} \le 2 \Rightarrow \frac{\log n}{b^i} \le 1$	$\log_b \log n$

1.4 Un primo esempio completo: la sottosequenza di somma massima

Problema (MaxSum). Dato $A = a_1, \ldots, a_n$ (anche negativi), trovare la massima somma di una sottosequenza **contigua** (quella vuota vale 0).

Ingenuo: provo ogni coppia (i, j) e risommo: $\Theta(n^3)$.
Meglio: a i fisso la somma si aggiorna con $A_i^{j+1} = A_i^j + a_{j+1}$: $\Theta(n^2)$.

Lineare (Kadane). Se scorrendo la somma corrente diventa negativa in r, conviene ripartire da $r + 1$: ogni sottosequenza che inizia in $[i, r]$ e finisce dopo r è battuta da quella che inizia in $r + 1$ (**lemma**: $A_z^q = A_z^r + A_{r+1}^q$ e $A_z^r \le A_i^r < 0$).

MAXSUM LINEARE — Θ(N)

```
1  MaxSum(A, n)
2    maxSum ← 0;  sum ← 0
3    for j ← 1 to n do
4      sum ← sum + A[j]
5      if sum < 0 then
6        sum ← 0                  // riparto da j+1
7      else if sum > maxSum then
8        maxSum ← sum
9    return maxSum
```

ESERCIZIO 1 DELLO SCRITTO

2 Algoritmi di ordinamento

Devi saper scrivere lo pseudocodice "di lezione" e soprattutto eseguirlo a mano senza sbagliare un passo. È l'esercizio più meccanico: sono punti sicuri.

COSA TI CHIEDONO

- Pseudocodice di 2 algoritmi (con le funzioni ausiliarie).
- L'array dopo certe iterazioni del ciclo più esterno.
- Una breve motivazione di ogni passo.

COSA DEVI SAPER FARE

- Sapere a quale valore dell'indice corrisponde la k -esima iterazione.
- Eseguire Heapify, Partiziona, gli spostamenti di Insertion Sort.
- Conoscere le complessità e perché.

2.1 Panoramica e "contare le iterazioni"

Algoritmo	Idea in una riga	Migliore	Medio	Peggiore	Sul posto
Insertion Sort	inserisco $A[j]$ nel prefisso già ordinato	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	sì
Selection Sort	porto il massimo in fondo, ripeto sul resto	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	sì
Merge Sort	divido a metà, ordino, fondo	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	no (array B)
Heap Sort	Selection Sort con uno heap: max in $O(\log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	sì
Quick Sort	partiziono attorno a un pivot, ricorro	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	sì

"AL TERMINE DELLA k -ESIMA ITERAZIONE DEL CICLO PIÙ ESTERNO"

Si contano le iterazioni a partire da **1**. Traduci sempre k nel valore dell'indice di ciclo **prima** di iniziare:

Ciclo	iterazione k ↔	esempio ($n = 12$)
InsertionSort: `for j ← 2 to n`	$j = k + 1$	3ª → $j = 4$; 7ª → $j = 8$; 10ª → $j = 11$
SelectionSort: `for i ← n downto 2`	$i = n - k + 1$	1ª → $i = 12$
BuildHeap (base 1): `for i ← ⌊n/2⌋ downto 1`	$i = \lfloor n/2 \rfloor - k + 1$	6 iterazioni: $i = 6, \ldots, 1$
BuildHeap (base 0): `for i ← ⌊n/2⌋-1 downto 0`	$i = \lfloor n/2 \rfloor - k$	6 iterazioni: $i = 5, \ldots, 0$
HeapSort (base 0): `for i ← n-1 downto 1`	$i = n - k$	2ª → $i = 10$; 5ª → $i = 7$; 8ª → $i = 4$
Partiziona: `repeat … until i ≥ j`	un giro del `repeat` esterno	finisce quando gli indici si incrociano

2.2 Insertion Sort

Idea (come le carte in mano): il prefisso $A[1..j-1]$ è già ordinato; prendo $x = A[j]$, sposto di un posto a destra tutti gli elementi del prefisso **maggiori di** x e metto x nel buco rimasto.

Inversioni. Una coppia (i, j) con $i < j$ e $a_i > a_j$. Ogni spostamento elimina esattamente un'inversione; una sequenza decrescente ne ha $\frac{n(n-1)}{2}$ ⇒ caso peggiore quadratico.

INSERTIONSORT ITERATIVO (DA RIPORTARE)

```
1 InsertionSort(A, n)
2   for j ← 2 to n do
3     x ← A[j]
4     i ← j - 1
5     while i > 0 and A[i] > x do
6       A[i+1] ← A[i]          // sposto a destra
7       i ← i - 1
8     A[i+1] ← x               // inserisco x
```

Analisi nei tre casi

Sia t_j = numero di volte in cui si valuta la condizione del `while` per un certo j ($1 \le t_j \le j$). Allora $T(n) = cn + c_4 \sum_{j=2}^{n} t_j + (c_5 + c_6) \sum_{j=2}^{n} (t_j - 1)$.

MIGLIORE

Array già ordinato: $t_j = 1$ ⇒ $T(n) = \Theta(n)$.

PEGGIORE

Array decrescente: $t_j = j$ ⇒ $\sum j = \Theta(n^2)$.

MEDIO

x finisce in ognuna delle j posizioni con prob. $\frac{1}{j}$: $E[t_j] = \frac{j+1}{2}$ ⇒ $\Theta(n^2)$.

Versione ricorsiva (vista nell'altro corso): `InsertionSort(A,n)`: se $n > 1$ ordina ricorsivamente $A[1..n-1]$ e poi inserisce $A[n]$ con lo stesso `while`.

Esame 16/06/2026 · 1(ii) InsertionSort su $A = [7, 6, 8, 5, 1, 9, 8, 0, 2, 4, 3, 1]$: array al termine della 3ª, 7ª e 10ª iterazione.

Iterazione $k \leftrightarrow j = k + 1$. Ecco **tutte** le iterazioni (le righe evidenziate sono quelle richieste):

prefisso ordinato $A[1..j]$ · dove è finito x · elementi spostati a destra

iter.	j	x=A[j]	1	2	3	4	5	6	7	8	9	10	11	12	cosa succede
–	–	–	7	6	8	5	1	9	8	0	2	4	3	1	array iniziale
1	2	6	6	7	8	5	1	9	8	0	2	4	3	1	6 va in pos. 1 (1 spostam.)
2	3	8	6	7	8	5	1	9	8	0	2	4	3	1	8 va in pos. 3 (resta lì)
3	4	5	5	6	7	8	1	9	8	0	2	4	3	1	5 va in pos. 1 (3 spostam.)
4	5	1	1	5	6	7	8	9	8	0	2	4	3	1	1 va in pos. 1 (4 spostam.)
5	6	9	1	5	6	7	8	9	8	0	2	4	3	1	9 va in pos. 6 (resta lì)
6	7	8	1	5	6	7	8	8	9	0	2	4	3	1	8 va in pos. 6 (1 spostam.)
7	8	0	0	1	5	6	7	8	8	9	2	4	3	1	0 va in pos. 1 (7 spostam.)
8	9	2	0	1	2	5	6	7	8	8	9	4	3	1	2 va in pos. 3 (6 spostam.)
9	10	4	0	1	2	4	5	6	7	8	8	9	3	1	4 va in pos. 4 (6 spostam.)
10	11	3	0	1	2	3	4	5	6	7	8	8	9	1	3 va in pos. 4 (7 spostam.)
11	12	1	0	1	1	2	3	4	5	6	7	8	8	9	1 va in pos. 3 (9 spostam.)

NEL COMPITO SCRIVI COSÌ

- **3ª iterazione** ($j = 4$, $x = 5$): 8, 7, 6 sono maggiori di 5 e scalano a destra; 5 va in posizione 1. → $[5, 6, 7, 8, 1, 9, 8, 0, 2, 4, 3, 1]$
- **7ª iterazione** ($j = 8$, $x = 0$): tutti i 7 elementi del prefisso sono > 0 e scalano; 0 va in posizione 1. → $[0, 1, 5, 6, 7, 8, 8, 9, 2, 4, 3, 1]$
- **10ª iterazione** ($j = 11$, $x = 3$): scalano 9, 8, 8, 7, 6, 5, 4 (tutti > 3), il while si ferma su $A[3] = 2 \leq 3$; 3 va in posizione 4. → $[0, 1, 2, 3, 4, 5, 6, 7, 8, 8, 9, 1]$

2.3 Selection Sort

Idea: trovo il **massimo** di $A[1..i]$ e lo scambio con $A[i]$; ora $A[i..n]$ è ordinato e definitivo. Ripeto con i da n fino a 2.

Costo: FindMax su i elementi costa $\Theta(i)$, sempre (non può fermarsi prima). Totale $\sum_{i=2}^{n} c\,i = \Theta(n^2)$ **in ogni caso**.

Il collo di bottiglia è trovare il massimo in tempo lineare: lo **heap** lo fa in $O(\log n)$ → Heap Sort.

SELECTIONSORT ITERATIVO (DA RIPORTARE)

```
1  SelectionSort(A, n)
2    for i ← n downto 2 do
3      j ← FindMax(A, i)
4      Swap(A, i, j)
5
6  FindMax(A, i)          // indice del max in A[1..i]
7    r ← 1
8    for k ← 2 to i do
9      if A[k] > A[r] then r ← k
10   return r
```

ALLENAMENTO: SELECTION SORT SULL'ARRAY DEL 14/07

parte finale già ordinata · posizione i appena fissata · da dove veniva il massimo

iter.	i	j max	1	2	3	4	5	6	7	8	9	10	11	12	cosa succede
–	–	–	3	2	1	0	8	1	2	5	2	7	3	9	array iniziale
1	12	12	3	2	1	0	8	1	2	5	2	7	3	9	max già in pos. 12
2	11	5	3	2	1	0	3	1	2	5	2	7	8	9	max in pos. 5 → scambio con 11
3	10	10	3	2	1	0	3	1	2	5	2	7	8	9	max già in pos. 10
4	9	8	3	2	1	0	3	1	2	2	5	7	8	9	max in pos. 8 → scambio con 9
5	8	1	2	2	1	0	3	1	2	3	5	7	8	9	max in pos. 1 → scambio con 8
6	7	5	2	2	1	0	2	1	3	3	5	7	8	9	max in pos. 5 → scambio con 7
7	6	1	1	2	1	0	2	2	3	3	5	7	8	9	max in pos. 1 → scambio con 6
8	5	2	1	2	1	0	2	2	3	3	5	7	8	9	max in pos. 2 → scambio con 5
9	4	2	1	0	1	2	2	2	3	3	5	7	8	9	max in pos. 2 → scambio con 4
10	3	1	1	0	1	2	2	2	3	3	5	7	8	9	max in pos. 1 → scambio con 3
11	2	1	0	1	1	2	2	2	3	3	5	7	8	9	max in pos. 1 → scambio con 2

Con il confronto stretto `A[k] > A[r]`, a parità di valore FindMax sceglie il **primo** massimo: nell'iterazione 5 ($i = 8$) il massimo 3 compare nelle posizioni 1 e 5, e viene scelta la 1.

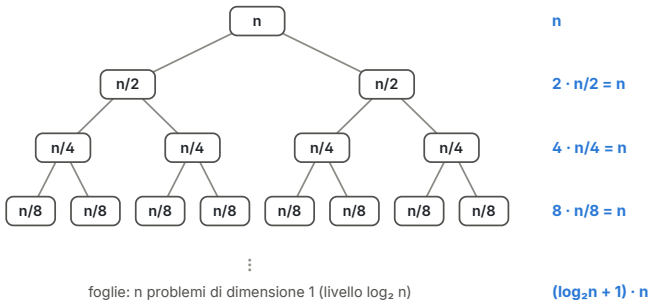
2.4 Merge Sort (divide et impera)

MERGESORT

```
1  MergeSort(A, p, r)
2    if p < r then
3      q ← ⌊(p + r)/2⌋
4      MergeSort(A, p, q)
5      MergeSort(A, q+1, r)
6      Merge(A, p, q, r)
7
8  Merge(A, p, q, r)     // fonde A[p..q] e A[q+1..r]
9    i ← p;  j ← q+1;  k ← p
10   while i ≤ q and j ≤ r do
11     if A[i] ≤ A[j] then B[k] ← A[i]; i ← i+1
12     else B[k] ← A[j]; j ← j+1
13     k ← k+1
14   if i ≤ q then z ← i else z ← j    // parte rimasta
15   while k ≤ r do B[k] ← A[z]; z ← z+1; k ← k+1
16   for i ← p to r do A[i] ← B[i]
```

Schema divide et impera: (1) se il problema è piccolo lo risolvo direttamente; (2) altrimenti lo divido in sottoproblemi, (3) li risolvo ricorsivamente e (4) combino le soluzioni.

Merge fa un passo per ogni elemento: $\Theta(n)$. Quindi $T(n) = 2T(n/2) + \Theta(n)$.



Ogni livello costa n; ci sono $\log_2 n + 1$ livelli $\Rightarrow \Theta(n \log n)$.

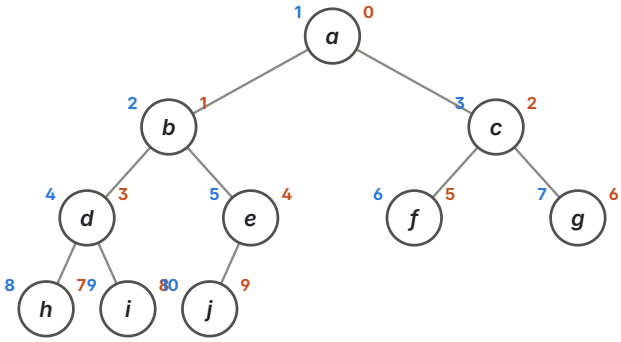
2.5 Heap e Heap Sort

Uno **heap** (max-heap) è un **albero binario completo** (tutti i livelli pieni tranne l'ultimo, riempito da sinistra) in cui **ogni padre è $\geq$ dei figli**. Quindi il massimo è nella radice e l'altezza è $\Theta(\log n)$.

Si memorizza in un array, livello per livello. Le formule dipendono dalla base degli indici:

	base 1 (lezione)	base 0 (tema 14/07)
figlio sinistro	$2i$	$2i+1$
figlio destro	$2i+1$	$2i+2$
padre	$\lfloor i/2 \rfloor$	$\lfloor (i-1)/2 \rfloor$
ultimo nodo interno	$\lfloor n/2 \rfloor$	$\lfloor n/2 \rfloor - 1$

Uno heap di n elementi ha $\lceil n/2 \rceil$ foglie: per questo BuildHeap parte dall'ultimo nodo interno.

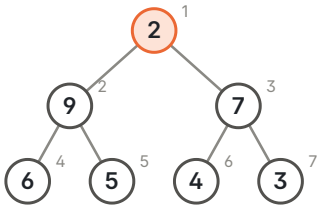


Indici **base 1** a sinistra, **base 0** a destra di ogni nodo.

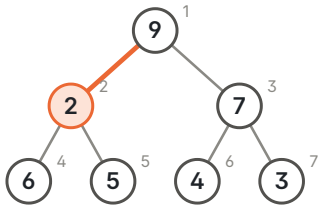


Heapify: "faccio scendere" un elemento

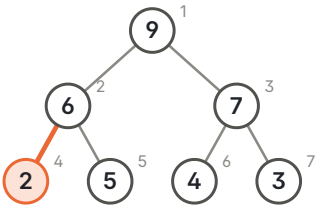
Precondizione: i sottoalberi dei figli di i sono già heap; solo $A[i]$ può essere fuori posto. Confronto $A[i]$ con i figli; se un figlio è più grande, scambio col **figlio maggiore** e continuo da lì. Costo $O(h)$ con h altezza del nodo, quindi $O(\log n)$.



A[1]=2 viola la proprietà



scambio col figlio maggiore (9)



scendo ancora: scambio con 6 → fatto

HEAP SORT — BASE 1 (LEZIONE)

```
Heapify(A, i)
  l ← 2i;  r ← 2i + 1
  if l ≤ heapsize and A[l] > A[i] then max ← l
  else max ← i
  if r ≤ heapsize and A[r] > A[max] then max ← r
  if max ≠ i then
    Swap(A, i, max)
    Heapify(A, max)

BuildHeap(A, n)
  heapsize ← n
  for i ← ⌊n/2⌋ downto 1 do
    Heapify(A, i)

HeapSort(A, n)
  BuildHeap(A, n)
  for i ← n downto 2 do
    Swap(A, 1, i)            // il max va in fondo
    heapsize ← heapsize - 1
    Heapify(A, 1)            // ripristino lo heap
```

HEAP SORT — BASE 0 (COME NEL TEMA DEL 14/07)

```
Heapify(A, n, i)             // n = dimensione heap
  l ← 2i + 1;  r ← 2i + 2;  max ← i
  if l < n and A[l] > A[max] then max ← l
  if r < n and A[r] > A[max] then max ← r
  if max ≠ i then
    Swap(A, i, max)
    Heapify(A, n, max)

BuildHeap(A, n)
  for i ← ⌊n/2⌋ - 1 downto 0 do
    Heapify(A, n, i)

HeapSort(A, n)
  BuildHeap(A, n)
  for i ← n - 1 downto 1 do
    Swap(A, 0, i)
    Heapify(A, i, 0)         // heap = A[0..i-1]
```

PERCHÉ BUILDHEAP COSTA Θ(N) E NON Θ(N LOG N)

Heapify su un nodo di altezza i costa i, e i nodi di altezza i sono circa $\frac{n}{2^{i+1}}$. Quindi

$$T_{BH}(n) \leq \sum_{i \geq 0} \frac{n}{2^{i+1}} i = \frac{n}{2} \sum_{i \geq 0} \frac{i}{2^i} = \frac{n}{2} \cdot 2 = n.$$

(La maggior parte dei nodi sta in basso, dove Heapify costa poco.)

COSTO DI HEAP SORT

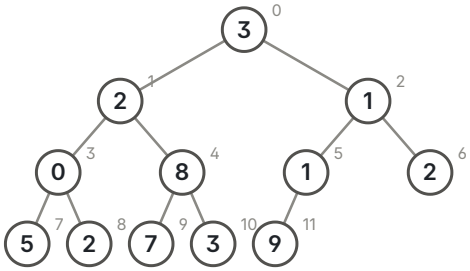
BuildHeap $\Theta(n)$ + $(n-1)$ Heapify sulla radice da $O(\log n)$ ⇒ $O(n \log n)$. Anche contando meglio (lo heap si accorcia) si ottiene $\approx n \log n - n$: $\Theta(n \log n)$, sul posto.

Esame 14/07/2026 · 1(ii)

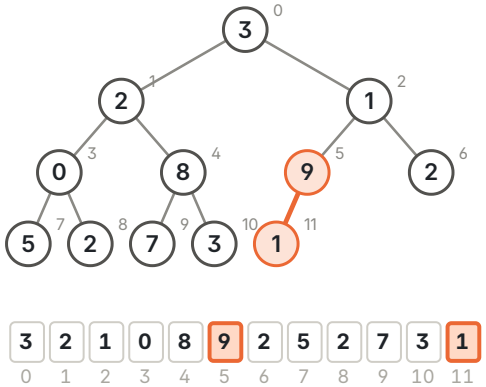
BuildHeap su $A = [3, 2, 1, 0, 8, 1, 2, 5, 2, 7, 3, 9]$ (indici 0...11): array al termine di ogni iterazione.

$n = 12$ ⇒ il ciclo va da $i = \lfloor 12/2 \rfloor - 1 = 5$ fino a 0: **6 iterazioni**. In arancione i nodi toccati e gli archi degli scambi.

Array iniziale

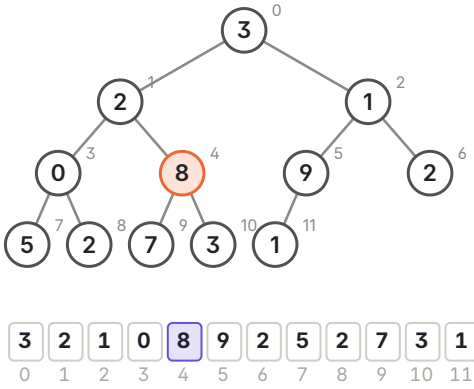


Iterazione 1: Heapify(A, 5)



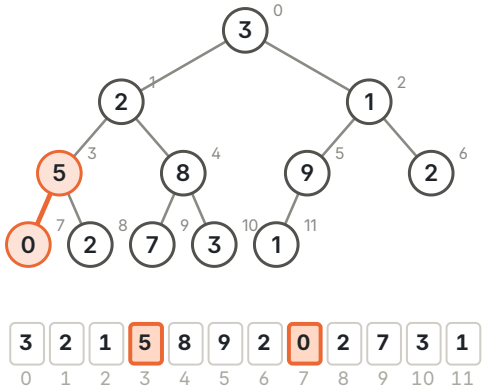
A[5]↔A[11]

Iterazione 2: Heapify(A, 4)



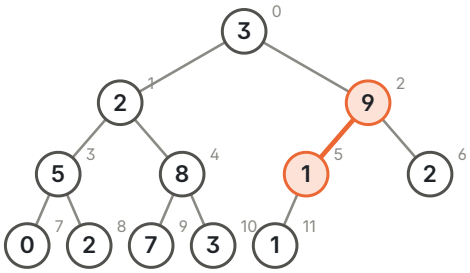
nessuno scambio

Iterazione 3: Heapify(A, 3)



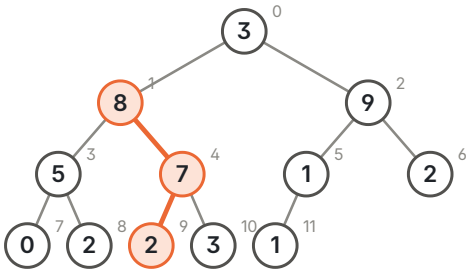
A[3]↔A[7]

Iterazione 4: Heapify(A, 2)



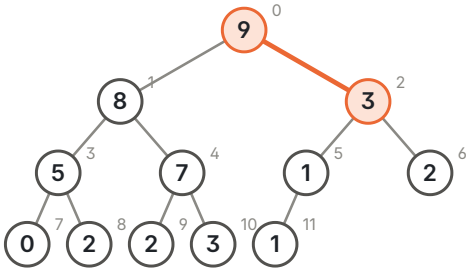
A[2]↔A[5]

Iterazione 5: Heapify(A, 1)



A[1]↔A[4], A[4]↔A[9]

Iterazione 6: Heapify(A, 0)



A[0]↔A[2]

NEL COMPITO SCRIVI COSÌ (UNA RIGA DI MOTIVAZIONE PER ITERAZIONE)

1. $i = 5$: figlio sinistro $A[11] = 9 > 1$ (non c'è il destro) ⇒ scambio → $[3, 2, 1, 0, 8, 9, 2, 5, 2, 7, 3, 1]$
2. $i = 4$: figli $A[9] = 7$, $A[10] = 3$, entrambi < 8 ⇒ nessuno scambio → invariato
3. $i = 3$: figli $A[7] = 5$, $A[8] = 2$; max $5 > 0$ ⇒ scambio $3 \leftrightarrow 7$ (pos. 7 è foglia) → $[3, 2, 1, 5, 8, 9, 2, 0, 2, 7, 3, 1]$
4. $i = 2$: figli $A[5] = 9$, $A[6] = 2$ ⇒ scambio $2 \leftrightarrow 5$; in pos. 5 l'1 ha figlio $A[11] = 1$, non maggiore → $[3, 2, 9, 5, 8, 1, 2, 0, 2, 7, 3, 1]$
5. $i = 1$: figli 5 e 8 ⇒ scambio $1 \leftrightarrow 4$; poi in pos. 4 il 2 ha figli 7 e 3 ⇒ scambio $4 \leftrightarrow 9$ → $[3, 8, 9, 5, 7, 1, 2, 0, 2, 2, 3, 1]$
6. $i = 0$: figli 8 e 9 ⇒ scambio $0 \leftrightarrow 2$; in pos. 2 il 3 ha figli 1 e 2 ⇒ stop → $[9, 8, 3, 5, 7, 1, 2, 0, 2, 2, 3, 1]$ ✔ max-heap

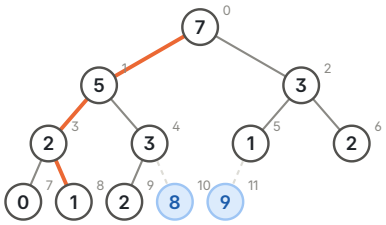
Esame 14/07/2026 · 1(iii)

HeapSort sullo stesso array: configurazione al termine della 2ª, 5ª e 8ª iterazione.

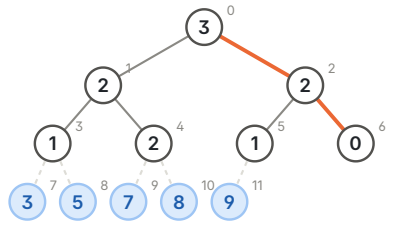
HeapSort prima esegue BuildHeap (risultato sopra), poi il ciclo `for i ← 11 downto 1`: iterazione $k \leftrightarrow i = 12 - k$. Ogni iterazione: **scambio** $A[0] \leftrightarrow A[i]$ (il massimo va in fondo, nella parte ordinata), poi **Heapify** su $A[0..i-1]$.

iter.	i	heapsize	0	1	2	3	4	5	6	7	8	9	10	11	dopo Swap(A,0,i) + Heapify(A,i,0)
–	–	–	9	8	3	5	7	1	2	0	2	2	3	1	heap dopo BuildHeap
1	11	11	8	7	3	5	3	1	2	0	2	2	1	9	scambi Heapify: 0↔1, 1↔4, 4↔10
2	10	10	7	5	3	2	3	1	2	0	1	2	8	9	scambi Heapify: 0↔1, 1↔3, 3↔8
3	9	9	5	3	3	2	2	1	2	0	1	7	8	9	scambi Heapify: 0↔1, 1↔4
4	8	8	3	2	3	1	2	1	2	0	5	7	8	9	scambi Heapify: 0↔1, 1↔3
5	7	7	3	2	2	1	2	1	0	3	5	7	8	9	scambi Heapify: 0↔2, 2↔6
6	6	6	2	2	2	1	0	1	3	3	5	7	8	9	scambi Heapify: 0↔1, 1↔4
7	5	5	2	1	2	1	0	2	3	3	5	7	8	9	scambi Heapify: 0↔1
8	4	4	2	1	0	1	2	2	3	3	5	7	8	9	scambi Heapify: 0↔2
9	3	3	1	1	0	2	2	2	3	3	5	7	8	9	scambi Heapify: nessuno
10	2	2	1	0	1	2	2	2	3	3	5	7	8	9	scambi Heapify: 0↔1
11	1	1	0	1	1	2	2	2	3	3	5	7	8	9	scambi Heapify: nessuno

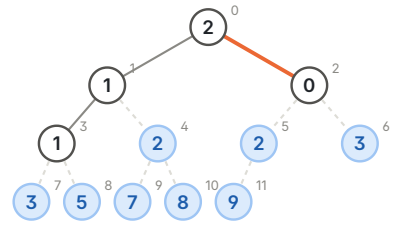
Fine iterazione 2 (i = 10)



Fine iterazione 5 (i = 7)



Fine iterazione 8 (i = 4)



RISPOSTA

- **2ª** ($i = 10$): scambio $A[0] = 8 \leftrightarrow A[10] = 1$; Heapify sullo heap $A[0..9]$ fa scendere l'1: scambio con il figlio 7 ($0 \leftrightarrow 1$), poi con 5 ($1 \leftrightarrow 3$), poi con 2 ($3 \leftrightarrow 8$) → $[7, 5, 3, 2, 3, 1, 2, 0, 1, 2 \mid 8, 9]$
- **5ª** ($i = 7$): scambio $A[0] = 3 \leftrightarrow A[7] = 0$; Heapify: lo 0 ha figli 2 e 3 ⇒ scambio $0 \leftrightarrow 2$; poi figli 1 e 2 ⇒ scambio $2 \leftrightarrow 6$ → $[3, 2, 2, 1, 2, 1, 0 \mid 3, 5, 7, 8, 9]$
- **8ª** ($i = 4$): scambio $A[0] = 2 \leftrightarrow A[4] = 0$; Heapify su $A[0..3]$: lo 0 ha figli 1 e 2 ⇒ scambio $0 \leftrightarrow 2$; in posizione 2 non ha figli nello heap ⇒ stop → $[2, 1, 0, 1 \mid 2, 2, 3, 3, 5, 7, 8, 9]$

ERRORI TIPICI SU HEAP

- Confondere base 0 e base 1 (figli $2i + 1, 2i + 2$ contro $2i, 2i + 1$): guarda gli indici scritti sopra l'array nel testo!
- Dimenticare che Heapify è **ricorsivo**: dopo uno scambio si continua a scendere.
- In HeapSort, Heapify lavora solo su $A[0..i - 1]$: gli elementi già messi in fondo non si toccano più.
- A parità di valori si scambia solo se il figlio è **strettamente** maggiore.

2.6 Quick Sort e Partiziona

Divide et impera "al contrario" di Merge Sort: il lavoro è nella **divisione**. Scelgo un **pivot** x e riorganizzo $A[p..r]$ in due parti contigue $A[p..q]$ e $A[q+1..r]$ tali che:

- **Requisito 1 (correttezza):** ogni elemento a sinistra è $\leq$ di ogni elemento a destra ⇒ niente fusione: ordinate le due parti, è tutto ordinato.
- **Requisito 2 (terminazione):** entrambe le parti sono **non vuote** ($p \leq q < r$) ⇒ le chiamate lavorano su input più piccoli.

QUICKSORT

```
1  QuickSort(A, p, r)
2    if p < r then
3      q ← Partiziona(A, p, r)
4      QuickSort(A, p, q)
5      QuickSort(A, q+1, r)
```

PARTIZIONA (SCHEMA DI HOARE, PIVOT = A[P])

```
1  Partiziona(A, p, r)
2    x ← A[p]                    // pivot
3    i ← p - 1
4    j ← r + 1
5    repeat
6      repeat j ← j - 1 until A[j] ≤ x
7      repeat i ← i + 1 until A[i] ≥ x
8      if i < j then Swap(A, i, j)
9    until i ≥ j
10   return j
```

COME "SI MUOVONO" I E J

j parte da destra e scende finché trova un elemento $\leq x$ (fuori posto a destra); i parte da sinistra e sale finché trova un elemento $\geq x$ (fuori posto a sinistra). Se non si sono incrociati li **scambio** e ricomincio. Quando $i \geq j$ mi fermo e restituisco j: $A[p..j]$ ha solo elementi $\leq x$, $A[j+1..r]$ solo elementi $\geq x$.

I cicli interni terminano sempre (al primo giro si fermano al più sul pivot; dopo uno scambio c'è sempre una "sentinella"). i e j si vengono incontro: $T_P(n) = \Theta(n)$.

Esame 16/06/2026 · 1(iii)

Partiziona su $A = [3, 2, 8, 5, 1, 9, 2, 0, 2, 7, 3, 1]$: array dopo ogni iterazione del ciclo esterno e partizioni finali.

$p = 1$, $r = 12$, pivot $x = A[1] = 3$, all'inizio $i = 0$, $j = 13$.

pivot / indici finali · elementi scambiati

iter.	i	j	1	2	3	4	5	6	7	8	9	10	11	12	cosa succede
–	–	–	**3**	2	8	5	1	9	2	0	2	7	3	1	pivot x = A[1] = 3; i = 0, j = 13
1	1	12	**1**	2	8	5	1	9	2	0	2	7	3	**3**	j si ferma in 12 (A[j] ≤ 3), i si ferma in 1 (A[i] ≥ 3): i < j ⇒ scambio A[1]↔A[12]
2	3	11	1	2	**3**	5	1	9	2	0	2	7	**8**	3	j si ferma in 11 (A[j] ≤ 3), i si ferma in 3 (A[i] ≥ 3): i < j ⇒ scambio A[3]↔A[11]
3	4	9	1	2	3	**2**	1	9	2	0	**5**	7	8	3	j si ferma in 9 (A[j] ≤ 3), i si ferma in 4 (A[i] ≥ 3): i < j ⇒ scambio A[4]↔A[9]
4	6	8	1	2	3	2	1	**0**	2	**9**	5	7	8	3	j si ferma in 8 (A[j] ≤ 3), i si ferma in 6 (A[i] ≥ 3): i < j ⇒ scambio A[6]↔A[8]
5	8	7	1	2	3	2	1	0	**2**	**9**	5	7	8	3	j si ferma in 7, i si ferma in 8: i ≥ j ⇒ **stop**, return j = 7

Partizioni create ($q = j = 7$):

$A[1..7]$: tutti ≤ 3 · $A[8..12]$: tutti ≥ 3

p						q=j					r
1	2	3	2	1	0	2	9	5	7	8	3
1	2	3	4	5	6	7	8	9	10	11	12

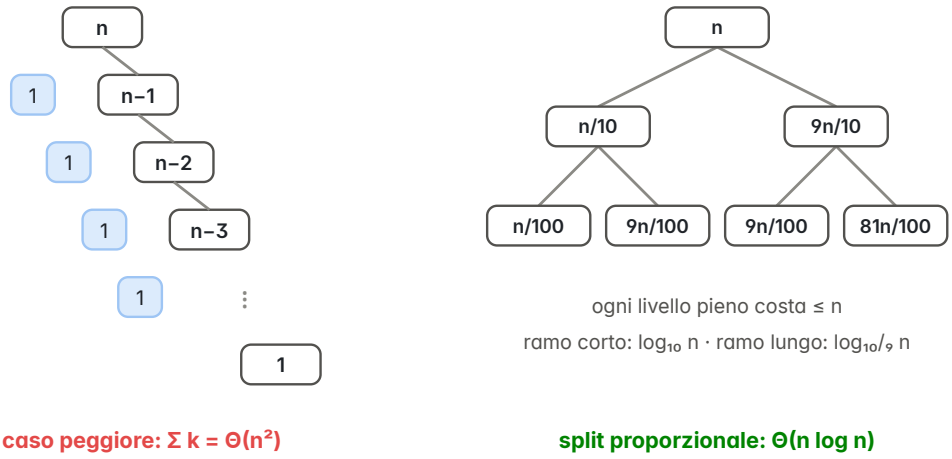
NEL COMPITO SCRIVI COSÌ

1. j scende da 13 a 12 ($A[12] = 1 \leq 3$), i sale a 1 ($A[1] = 3 \geq 3$); $1 < 12$ ⇒ scambio → $[1, 2, 8, 5, 1, 9, 2, 0, 2, 7, 3, 3]$
2. j si ferma in 11 ($3 \leq 3$), i in 3 ($8 \geq 3$) ⇒ scambio → $[1, 2, 3, 5, 1, 9, 2, 0, 2, 7, 8, 3]$
3. j salta 7 e si ferma in 9 (2), i in 4 (5) ⇒ scambio → $[1, 2, 3, 2, 1, 9, 2, 0, 5, 7, 8, 3]$
4. j si ferma in 8 (0), i salta 1 e si ferma in 6 (9) ⇒ scambio → $[1, 2, 3, 2, 1, 0, 2, 9, 5, 7, 8, 3]$
5. j si ferma in 7 (2), i in 8 (9): $i \geq j$ ⇒ nessuno scambio, **fine**, restituisce $q = 7$.

Partizioni: $A[1..7] = [1, 2, 3, 2, 1, 0, 2]$ (elementi ≤ 3) e $A[8..12] = [9, 5, 7, 8, 3]$ (elementi ≥ 3). Entrambe non vuote, come richiesto dal Requisito 2.

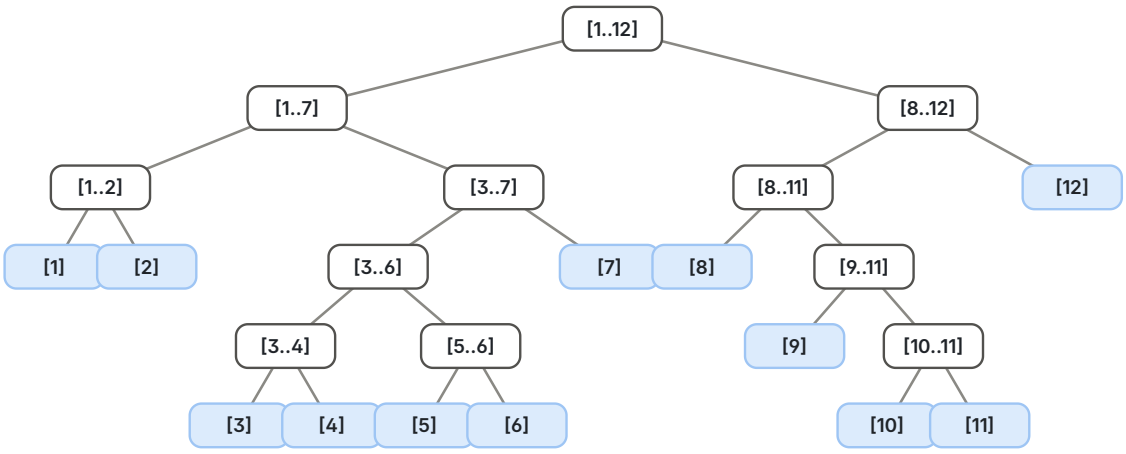
Analisi di Quick Sort

Con $Q = q - p + 1$ la dimensione della parte sinistra: $T(n) = \Theta(n) + T(Q) + T(n - Q)$.



Sinistra: partizioni sempre 1 / $n - 1$ (es. array già ordinato, pivot = minimo). Destra: partizioni proporzionali $\frac{1}{10}/\frac{9}{10}$.

- **Peggiore** (sbilanciamento massimo, $Q = 1$): $T(n) = T(n-1) + \Theta(n) = \sum_{k=1}^{n} \Theta(k) = \Theta(n^2)$.
- **Migliore** (sempre a metà): come Merge Sort, $\Theta(n \log n)$.
- **Proporzionale** (anche $\frac{1}{1000}$ / $\frac{999}{1000}$): l'albero sta tra uno pieno alto $\log_{10} n$ e uno alto $\log_{10/9} n$, ogni livello costa $\leq n$ ⇒ $\Theta(n \log n)$.
- **Medio** (elementi distinti, pivot con rango uniforme): $T(n) = \frac{2}{n} \sum_{q=1}^{n-1} T(q) + \Theta(n)$. Per induzione $T(n) \leq c\, n \log n$ usando $\sum_{q=1}^{n-1} q \log q \leq \frac{n^2}{2} \log n - \frac{n^2}{8}$ (spezzando la somma a metà) e scegliendo $c \geq 4k$ ⇒ $\Theta(n \log n)$.



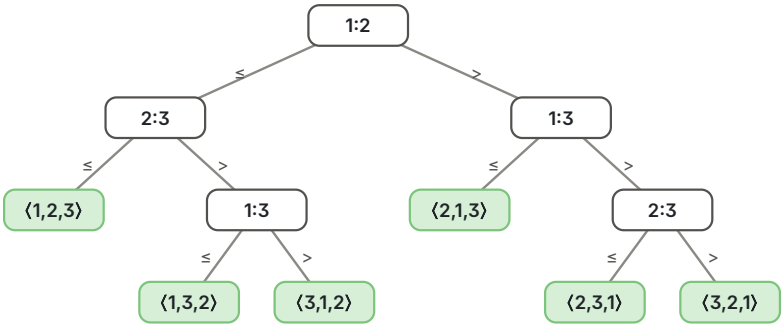
Bonus: tutte le chiamate di QuickSort sull'array del 16/06 (azzurro: casi base).

2.7 Limite inferiore: nessuno fa meglio di $n \log n$

Un algoritmo che ordina **per confronti** si descrive, fissato n, con un **albero di decisione**: ogni nodo interno è un confronto "$i : j$", ogni foglia una permutazione; un'esecuzione è un cammino radice → foglia.

- Servono almeno $n!$ foglie (una per ogni ordine possibile).
- Un albero binario di altezza h ha al più 2^h foglie ⇒ $h \geq \log(n!)$.
- $n! \geq (n/2)^{n/2}$ ⇒ $h \geq \frac{n}{2} \log \frac{n}{2} = \Omega(n \log n)$.

Medio: l'albero che minimizza il percorso esterno ha le foglie solo sugli ultimi due livelli (argomento di scambio); la profondità media è $\approx \log n! = \Omega(n \log n)$. **Migliore:** ogni cammino deve confrontare gli elementi consecutivi ⇒ almeno $n - 1$ confronti: $\Omega(n)$ (Insertion Sort lo raggiunge).



Albero di decisione di Insertion Sort per $n = 3$: 6 foglie, altezza 3.

CONSEGUENZA

Merge Sort, Heap Sort e (in media) Quick Sort sono **asintoticamente ottimi** tra gli algoritmi per confronti.

CHECKLIST DELL'ESERCIZIO 1

- Pseudocodice **completo** di tutte le funzioni richieste (es. HeapSort **e** BuildHeap **e** Heapify).
- Prima di simulare: base degli indici, n, e corrispondenza "iterazione k ↔ valore dell'indice".
- Per ogni configurazione: una riga di **motivazione** (chi si confronta, chi si scambia, perché si ferma).
- Ricontrolla che l'array finale sia una **permutazione** di quello iniziale (stessi elementi!).

ESERCIZIO 2 DELLO SCRITTO

3 Equazioni di ricorrenza con l'albero di ricorsione

Un unico metodo, sempre gli stessi 5 passi: albero → tabella dei costi → serie → altezza → risultato. Il prof accetta **solo** questo metodo.

COSA TI CHIEDONO

- "Si risolva l'equazione calcolandone l'**andamento asintotico**, utilizzando il **metodo degli alberi di ricorrenza**."

COSA DEVI SAPER FARE

- Disegnare i primi 2–3 livelli e trovare il costo del livello i.
- Sommare una serie geometrica e calcolare le altezze h_{min}, h_{max}.
- Maneggiare logaritmi e potenze.

3.1 L'idea: il tempo è la somma dei nodi dell'albero

Eseguire un algoritmo ricorsivo genera un **albero di ricorsione**: ogni nodo è una chiamata, i figli sono le chiamate ricorsive che fa. Ogni nodo x ha un **contributo locale** $C(x)$: le operazioni che la chiamata esegue *escluse* le chiamate ricorsive. Allora

$$T(n) = \sum_{x \in \text{nodi}} C(x).$$

Invece di sommare nodo per nodo, sommiamo **livello per livello**: prima il costo totale di ogni livello, poi la somma dei livelli (una serie).

? COME SI LEGGE $T(n) = a\,T(g(n)) + f(n)$

- $g(n)$ = **dimensione** dei figli (es. $\frac{n}{2}$, $\sqrt{n}$).
- a = **molteplicità**: quanti figli di quel tipo (si scrive sull'arco).
- $f(n)$ = **termine noto** = costo locale di un nodo di dimensione n.
- Il caso base dice quando l'albero si ferma.

3.2 Il metodo in 5 passi

1. **Disegna l'albero.** Livello 0: un nodo di dimensione n. Ogni nodo di dimensione m genera figli di dimensione $g(m)$: sostituisci m, non n (es. il figlio di $n^{1/2}$ in $T(n^{1/2})$ è $(n^{1/2})^{1/2} = n^{1/4}$). Scrivi la **molteplicità sull'arco** (il prof suggerisce di "sbarrare" la linea e scriverci il coefficiente). Bastano 2–3 livelli.

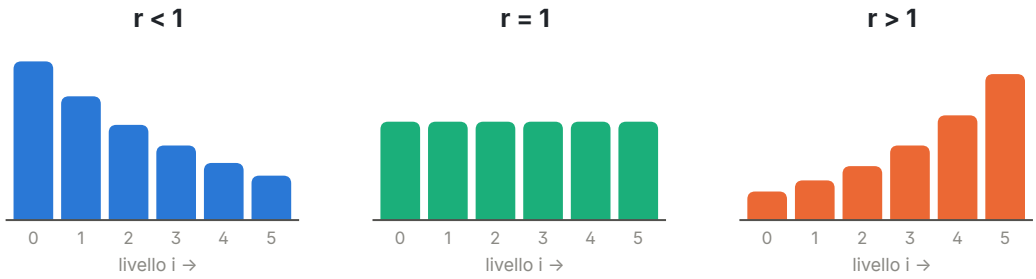
2. **Compila la tabella** con 3 colonne: *Livello* · *Costo locale* (il termine noto f calcolato sulla dimensione dei nodi di quel livello; nodi diversi separati da ";", ognuno scritto una volta) · *Costo totale del livello* (numero di nodi × costo locale, sommato su tutti i tipi di nodo).

3. **Trova il termine generale** del livello i e la **ragione** r (il numero elevato alla i). La somma dei livelli è una serie: $\sum_{i=0}^{k}(\text{costo livello } i)$, sommata con le formule della serie geometrica fino a un'altezza generica k.

4. **Calcola l'altezza**: imponi "dimensione al livello i ≤ caso base" e ricava i. Se tutti i rami scendono alla stessa velocità c'è un'unica altezza h. Se l'albero è **storto**: il ramo che scende **più velocemente** dà h_{min}, quello che scende **più lentamente** dà h_{max} (usa la dimensione **senza** il termine noto).

5 **Sostituisci e concludi.** Un'altezza: $T(n) = \Theta(\text{serie fino a } h)$. Due altezze: $\Omega(\text{serie fino a } h_{min}) \leq T(n) \leq O(\text{serie fino a } h_{max})$ (fino a h_{min} l'albero è pieno, oltre h_{max} non c'è più niente). Se $r < 1$ la serie converge e le altezze non servono: $T(n) = \Theta(\text{costo del livello } 0)$.

3.3 La ragione r decide tutto



domina il livello 0 (la radice) — **Θ(costo radice)**

tutti i livelli costano uguale — **Θ(costo livello × altezza)**

dominano gli ultimi livelli — **Θ(r^h · costo radice)**

Costo dei livelli (barre) al variare della ragione: chi "pesa di più" determina l'andamento.

✅ $r < 1$

$\sum_{i \geq 0} r^i = \frac{1}{1-r}$ costante ⇒ $T(n) = \Theta(f(n))$, il costo della radice.

✅ $r = 1$

Ogni livello costa uguale ⇒ $T(n) = \Theta(f(n) \cdot h)$. Es. Merge Sort: $n \cdot \log n$.

✅ $r > 1$

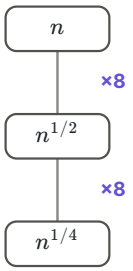
$\sum_{i=0}^{h} r^i = \frac{r^{h+1}-1}{r-1} = \Theta(r^h)$ ⇒ serve l'altezza; usa $r^{\log_b x} = x^{\log_b r}$.

Gli esercizi d'esame rientrano in tre **casi**, a seconda della forma dell'albero:

Caso	Forma	Esempio	Particolarità
1	un ramo, molteplicità **costante**	$8\,T(\sqrt{n}) + \log^2 n$	un'unica altezza
2	un ramo, molteplicità **non costante**	$3\sqrt[3]{n^2}\,T(\sqrt[3]{n}) + 7n$ 14/07	il numero di nodi dipende da n: moltiplica le molteplicità livello per livello
3	**due rami** (molteplicità costanti)	$T(\frac{3n}{5}) + 4T(\frac{n}{2}) + n$ 16/06	albero storto: h_{min} e h_{max}

3.4 Caso 1 — un ramo, molteplicità costante

Esempio · Gennaio 2026 $T(n) = 1$ se $n \leq 2$; $T(n) = 8\,T(\sqrt{n}) + \log^2 n$ altrimenti.



liv. 0: $\log^2 n$

liv. 1: $8 \cdot \frac{1}{4} \log^2 n = 2 \log^2 n$

liv. 2: $64 \cdot \frac{1}{16} \log^2 n = 4 \log^2 n$

$\vdots$

liv. i: $2^i \log^2 n$

Liv.	Costo locale	Costo totale
0	$\log^2 n$	$\log^2 n$
1	$\log^2 n^{1/2} = \frac{1}{4} \log^2 n$	$8 \cdot \frac{1}{4} \log^2 n = 2 \log^2 n$
2	$\log^2 n^{1/4} = \frac{1}{16} \log^2 n$	$64 \cdot \frac{1}{16} \log^2 n = 4 \log^2 n$
i	$\frac{1}{4^i} \log^2 n$	$8^i \cdot \frac{1}{4^i} \log^2 n = 2^i \log^2 n$

Serie ($r = 2$): $\sum_{i=0}^{k} 2^i \log^2 n = \log^2 n\,(2^{k+1} - 1)$. **Altezza** (albero bilanciato): $n^{1/2^i} \leq 2 \iff \frac{\log_2 n}{2^i} \leq 1 \iff i \geq \log_2 \log_2 n \Rightarrow h = \log_2 \log_2 n$.

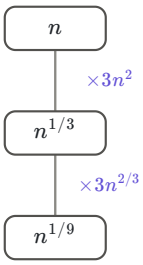
Risultato: $T(n) = \Theta\big(\log^2 n\,(2^{\log_2 \log_2 n + 1} - 1)\big) = \Theta\big(\log^2 n\,(2 \log_2 n - 1)\big) = \boxed{\Theta(\log^3 n)}$.

3.5 Caso 2 — un ramo, molteplicità non costante

IL TRUCCO

Se un nodo di dimensione m ha $a(m)$ figli (es. $3m^2$), il numero di nodi al livello i è il **prodotto** delle molteplicità calcolate sulle dimensioni dei livelli precedenti: $N_i = a(n) \cdot a(g(n)) \cdots a(g^{i-1}(n))$. Scrivi i primi livelli **per esteso** e guarda come si semplificano gli esponenti.

Esempio · dalle dispense $T(n) = 1$ se $n \leq 27$; $T(n) = 3n^2\,T(n^{1/3}) + 2n^3$.



liv. 0: $2n^3$

liv. 1: $3n^2 \cdot 2n = 6n^3$

liv. 2: $3n^2 \cdot 3n^{2/3} \cdot 2n^{1/3} = 18n^3$

$\vdots$

liv. i: $2 \cdot 3^i n^3$

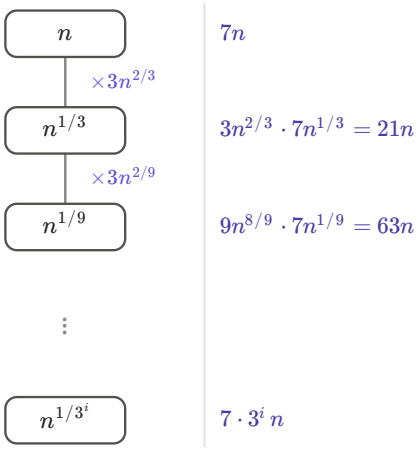
Livello 1: $3n^2$ nodi di dimensione $n^{1/3}$, ognuno costa $2(n^{1/3})^3 = 2n \Rightarrow 6n^3$.
Livello 2: $3n^2 \cdot 3(n^{1/3})^2 = 9n^{8/3}$ nodi, ognuno $2(n^{1/9})^3 = 2n^{1/3} \Rightarrow 18n^3$.
Livello i: $2 \cdot 3^i n^3$ ($r = 3$).

Serie: $\sum_{i=0}^{k} 2 \cdot 3^i n^3 = n^3(3^{k+1} - 1)$. Altezza: $n^{1/3^i} \leq 27 \iff \log_3 n \leq 3^{i+1} \iff i \geq \log_3 \log_3 n - 1$.

$\Rightarrow T(n) = \Theta\big(n^3(\log_3 n - 1)\big) = \boxed{\Theta(n^3 \log n)}$.

Esame 14/07/2026 · Es. 2 $T(n) = 1$ se $n \le 3$; $T(n) = 3\sqrt[3]{n^2}\,T(\sqrt[3]{n}) + 7n$ altrimenti.

1. Albero. Un solo tipo di figlio: un nodo di dimensione m ha $3m^{2/3}$ figli di dimensione $m^{1/3}$ e costo locale $7m$.



2. Tabella.

Liv.	n° nodi	Costo loc.	Costo totale
0	1	$7n$	$7n$
1	$3n^{2/3}$	$7n^{1/3}$	$21n$
2	$3n^{\frac{2}{3}} \cdot 3(n^{\frac{1}{3}})^{\frac{2}{3}} = 9n^{8/9}$	$7n^{1/9}$	$63n$
i	$3^i\, n^{1-1/3^i}$	$7n^{1/3^i}$	$7 \cdot 3^i\, n$

Esponente dei nodi: $\frac{2}{3}\left(1 + \frac{1}{3} + \cdots + \frac{1}{3^{i-1}}\right) = \frac{2}{3} \cdot \frac{1-3^{-i}}{1-\frac{1}{3}} = 1 - \frac{1}{3^i}$. Moltiplicato per il costo $n^{1/3^i}$ dà proprio n.

3. Serie (ragione $r = 3 > 1$): $\displaystyle\sum_{i=0}^{k} 7 \cdot 3^i n = 7n\,\frac{3^{k+1}-1}{3-1} = \frac{7n}{2}\left(3^{k+1}-1\right)$.

4. Altezza (un solo ramo ⇒ unica altezza): $n^{1/3^i} \le 3 \iff \frac{1}{3^i}\log_3 n \le 1 \iff \log_3 n \le 3^i \iff i \ge \log_3 \log_3 n$ ⇒ $h = \log_3 \log_3 n$.

5. Risultato: $T(n) = \frac{7n}{2}\left(3^{\log_3 \log_3 n + 1} - 1\right) = \frac{7n}{2}\left(3\log_3 n - 1\right) = \boxed{\Theta(n \log n)}$.

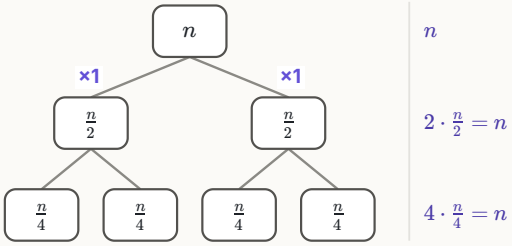
> ✍ **NEL COMPITO SCRIVI COSÌ**
> Albero (2–3 livelli con molteplicità $3n^{2/3}$, $3n^{2/9}$ sugli archi) → tabella con livello i: $7 \cdot 3^i n$ → serie $\frac{7n}{2}(3^{k+1}-1)$ → altezza $h = \log_3 \log_3 n$ (albero bilanciato: $h_{min} = h_{max}$) → $T(n) = \frac{7n}{2}(3\log_3 n - 1) = \Theta(n \log n)$.

3.6 Caso 3 — due rami

Ora ogni nodo ha figli di **due tipi**. Al livello 2 compaiono anche nodi "misti" (es. $\sqrt{n/3}$ o $\frac{3n}{10}$): contali tutti, ogni combinazione con la sua molteplicità (il prodotto delle molteplicità lungo il cammino). Due esempi rapidi e poi l'esame.

MERGE SORT (CASO R = 1)

$T(n) = 2T(\frac{n}{2}) + n$



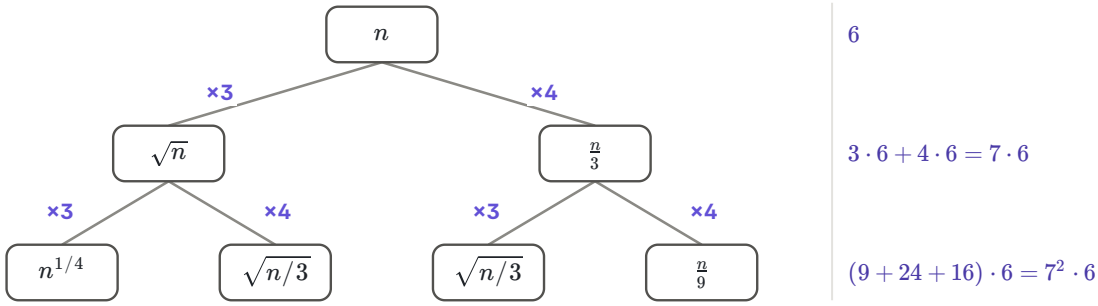
Ogni livello costa n; altezza $\frac{n}{2^i} \leq 1 \iff i \geq \log_2 n$.
⇒ $T(n) = \sum_{i=0}^{\log n} n = n(\log_2 n + 1) = \Theta(n \log n)$.

MARZO 2026 (CASO R < 1)

$T(n) = 2T(\sqrt{n}) + 3T(\sqrt[3]{n}) + \log^2 n$, $T(n) = 1$ se $n \leq 2$.
Livello 1: $2(\log n^{1/2})^2 + 3(\log n^{1/3})^2 = \frac{2}{4}\log^2 n + \frac{3}{9}\log^2 n = \frac{5}{6}\log^2 n$; livello i: $(\frac{5}{6})^i \log^2 n$.

$r = \frac{5}{6} < 1$: $\sum_{i\geq 0}(\frac{5}{6})^i \log^2 n = 6\log^2 n$. Le altezze ($h_{min} = \log_3 \log_2 n$, $h_{max} = \log_2 \log_2 n$) non servono: entrambi i limiti danno $6\log^2 n \Rightarrow \Theta(\log^2 n)$.

Esempio · Febbraio 2026 $T(n) = 1$ se $n \leq 3$; $T(n) = 3\,T(\sqrt{n}) + 4\,T(\frac{n}{3}) + 6$.



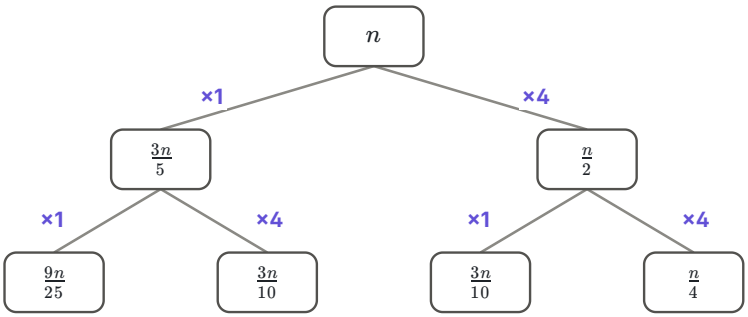
Il costo locale è costante (6): livello i = (numero nodi) $\cdot 6 = 7^i \cdot 6$. Serie: $6\sum_{i=0}^{k} 7^i = 7^{k+1} - 1$.
h_{min} (ramo $\sqrt{}$, il più veloce): $n^{1/2^i} \leq 3 \Rightarrow h_{min} = \log_2 \log_3 n$.
h_{max} (ramo $\frac{n}{3}$, il più lento): $\frac{n}{3^i} \leq 3 \Rightarrow h_{max} \approx \log_3 n$.

$7^{\log_2 \log_3 n+1} - 1 \leq T(n) \leq 7^{\log_3 n+1} - 1 \Rightarrow$

$$\boxed{\Omega\left((\log_3 n)^{\log_2 7}\right) \leq T(n) \leq O\left(n^{\log_3 7}\right)}$$

Esame 16/06/2026 · Es. 2 $T(n) = 1$ se $n \le 1$; $T(n) = T(\frac{3n}{5}) + 4\,T(\frac{n}{2}) + n$ altrimenti.

1. Albero. Ogni nodo di dimensione m ha **1** figlio di dimensione $\frac{3m}{5}$ e **4** figli di dimensione $\frac{m}{2}$; costo locale m.



2. Tabella.

Liv.	Costo locale	Costo totale
0	n	n
1	$\frac{3n}{5}$; $\frac{n}{2}$	$1 \cdot \frac{3n}{5} + 4 \cdot \frac{n}{2} = \frac{13}{5}n$
2	$\frac{9n}{25}$; $\frac{3n}{10}$; $\frac{n}{4}$	$\frac{9n}{25} + 8 \cdot \frac{3n}{10} + 16 \cdot \frac{n}{4} = \frac{169}{25}n$
i	…	$(\frac{13}{5})^i n$

Livello 2: i nodi $\frac{3n}{10}$ sono $1 \cdot 4 + 4 \cdot 1 = 8$ (sinistro→destro e destro→sinistro); i nodi $\frac{n}{4}$ sono $4 \cdot 4 = 16$. Controllo: $\frac{9}{25} + \frac{24}{10} + 4 = \frac{169}{25} = (\frac{13}{5})^2$ ✔

3. Serie ($r = \frac{13}{5} > 1$):

$$\sum_{i=0}^{k} (\tfrac{13}{5})^i n = n\,\frac{(\frac{13}{5})^{k+1} - 1}{\frac{13}{5} - 1} = \frac{5n}{8}\left[(\tfrac{13}{5})^{k+1} - 1\right]$$

4. Altezze (albero storto):

- h_{min}, ramo più veloce $\frac{n}{2^i}$: $\frac{n}{2^i} \le 1 \iff i \ge \log_2 n$ ⇒ $h_{min} = \log_2 n$.
- h_{max}, ramo più lento $(\frac{3}{5})^i n$: $(\frac{3}{5})^i n \le 1 \iff (\frac{5}{3})^i \ge n \iff i \ge \log_{5/3} n$ ⇒ $h_{max} = \log_{5/3} n$.

5. Risultato. Con lo scambio $a^{\log_b n} = n^{\log_b a}$: $(\frac{13}{5})^{\log_2 n} = n^{\log_2 \frac{13}{5}}$ e $(\frac{13}{5})^{\log_{5/3} n} = n^{\log_{5/3} \frac{13}{5}}$. Quindi

$$\frac{5n}{8}\left[\tfrac{13}{5}\, n^{\log_2 \frac{13}{5}} - 1\right] \le T(n) \le \frac{5n}{8}\left[\tfrac{13}{5}\, n^{\log_{5/3} \frac{13}{5}} - 1\right]$$

RISPOSTA

$$T(n) = \Omega\left(n^{1+\log_2 \frac{13}{5}}\right) \approx \Omega(n^{2.38}) \qquad \text{e} \qquad T(n) = O\left(n^{1+\log_{5/3} \frac{13}{5}}\right) \approx O(n^{2.87})$$

Equivalentemente $n^{\log_2 \frac{26}{5}}$ e $n^{\log_{5/3} \frac{13}{3}}$. Con due altezze diverse il metodo dà una **forbice** $\Omega(\dots) \le T(n) \le O(\dots)$: è la risposta attesa (come nell'esempio di febbraio).

ERRORI TIPICI

- Sostituire n invece della **dimensione del nodo** nel livello successivo ($T(n^{1/2})$ ha figli $n^{1/4}$, non $n^{1/2}$).
- Dimenticare la molteplicità dei nodi "misti" al livello 2 nel caso a due rami.
- Nel caso 2, moltiplicare per $3n^{2/3}$ a **ogni** livello: la molteplicità va calcolata sulla dimensione del nodo ($3(n^{1/3})^{2/3}$ al livello 2).
- Scambiare h_{min} e h_{max}: il ramo che **scende più in fretta** finisce prima ⇒ dà l'altezza **minima**.
- Con $r > 1$ non fermarsi alla serie: va sostituita l'altezza e semplificata con $a^{\log_b c} = c^{\log_b a}$.

ESERCIZIO 3 DELLO SCRITTO

4 Da algoritmo ricorsivo ad algoritmo iterativo

"Si scriva un algoritmo iterativo che simuli precisamente l'algoritmo ricorsivo": devi rifare a mano quello che fa lo stack di sistema, con lo schema visto a lezione.

COSA TI CHIEDONO

- Una funzione ricorsiva (quasi sempre su un **albero binario**) con **due chiamate ricorsive**, funzioni esterne, variabili locali.
- Una versione iterativa che esegua le **stesse operazioni nello stesso ordine**.

COSA DEVI SAPER FARE

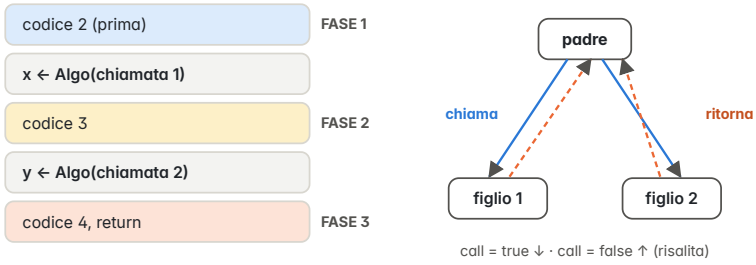
- Scegliere quali variabili salvare (stack) e come scegliere **last**.
- Scrivere il ciclo con **call**, ripristino (Top), Push e Pop al momento giusto.

4.1 Cosa fa davvero una chiamata ricorsiva

Ogni volta che una funzione chiama sé stessa, il linguaggio **salva il contesto** (variabili locali e punto in cui riprendere) sullo stack di sistema, esegue la chiamata e al ritorno **ripristina** il contesto. L'esecuzione è una **visita dell'albero di ricorsione**: si scende con le chiamate e si risale con i return.

Il corpo di ogni chiamata è diviso in **fasi** dalle chiamate ricorsive:

- **Fase 1**: dall'inizio fino alla chiamata 1 (oppure, nel caso base, fino al return);
- **Fase 2**: dal ritorno della chiamata 1 fino alla chiamata 2;
- **Fase 3**: dal ritorno della chiamata 2 fino alla fine.



Le tre fasi di una chiamata e i due "movimenti" nell'albero.

4.2 Gli ingredienti dello schema

Variabile	A cosa serve
`call`	**true** quando sto **entrando** in una nuova chiamata (Fase 1); **false** quando sto **risalendo** da una chiamata finita (Fasi 2 e 3). Negli appunti si chiama anche `start`.
`ret`	simula il valore restituito dalla chiamata appena terminata.
`last`	ricorda un **parametro dell'ultima chiamata terminata**: tornando nel padre, dice se sto tornando dalla chiamata 1 o dalla 2.
uno **stack per ogni variabile** da salvare	stack "paralleli" (es. `ST`, `Si`, `Sa`), spinti e svuotati insieme. Il ciclo continua finché `call` è vero o c'è ancora una chiamata sospesa.

IL PROF NON ACCETTA

Uno stack di "stato/fase" al posto di `last`, oppure un unico stack di record ("frame") al posto degli stack paralleli: secondo quanto riferito a ricevimento, viene considerato uno spreco e l'esercizio è annullato. Usa **call + last + stack paralleli**.

4.3 Le tre regole

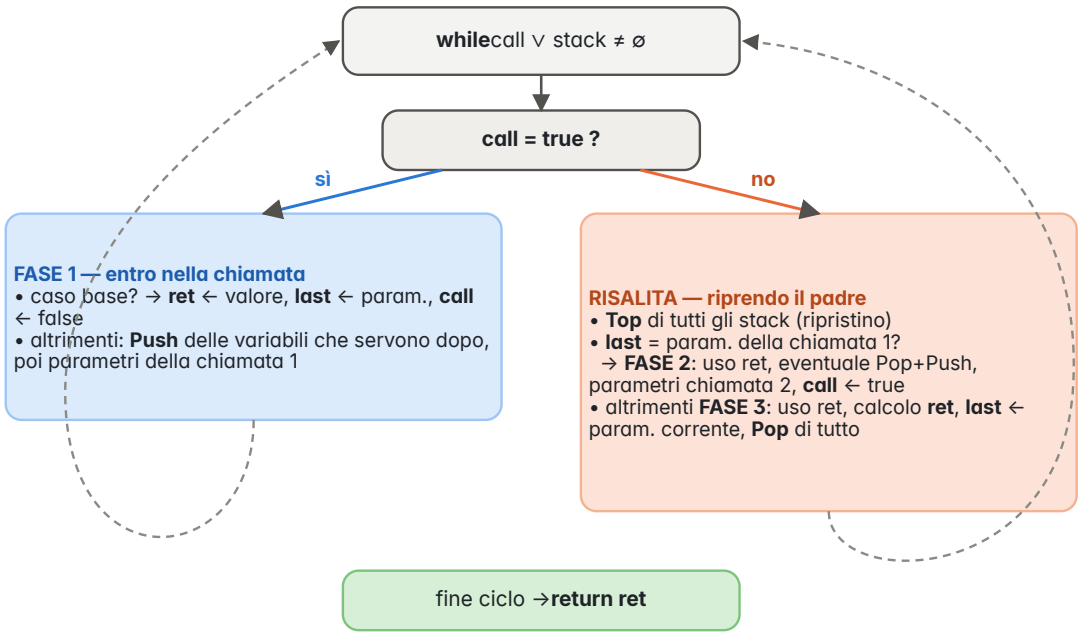
1. **Quali variabili vanno su uno stack?** Quelle che **esistono prima** di una chiamata ricorsiva, **vengono usate dopo** di essa e verrebbero **modificate** dalla chiamata (perché la chiamata usa la stessa variabile per sé). Tipicamente: il nodo T, i parametri usati dopo (es. i, b), le variabili locali calcolate prima e lette dopo (es. a, l).
   Se una variabile viene **sovrascritta** prima di essere letta (es. $r \leftarrow$ 1° risultato, poi $r \leftarrow$ 2° risultato) **non** serve lo stack: il suo vecchio valore non serve più.

2. **Come scelgo `last`?** Un parametro che ha **sempre valori diversi** nelle due chiamate. Si assegna nei **casi base** e alla **fine della Fase 3**, con il valore del parametro della chiamata che sta terminando. Nel padre si confronta con il valore che quel parametro ha nella chiamata 1: uguale ⇒ è finita la 1, altrimenti la 2.

3. **Quando Push, Top e Pop?** **Push** quando "chiamo" la chiamata 1 (salvo ciò che servirà dopo). **Top** sempre all'**inizio della risalita** (ripristino il contesto del padre). **Pop** alla fine della Fase 3 (la chiamata è finita), oppure Pop+Push per **aggiornare** un valore salvato che cambia nella Fase 2 (o per salvarne uno nuovo prima della chiamata 2).

Chiamate 1 e 2	last	Funziona?
`A(T→sx, 3·i)` e `A(T→dx, 3·i+2)`	i della chiamata che termina; nel padre: `last = 3·i`?	**sì** $3i \neq 3i+2$ sempre
`MS(A,p,q)` e `MS(A,q+1,r)`	estremo sinistro; nel padre: `last = p`?	**sì** $p \neq q+1$
`Max(A,p,q)` e `Max(A,q+1,r)`	estremo destro; nel padre: `last ≠ r` ⇒ era la 1	**sì** $q < r$
`A(T→sx, …)` e `A(T→dx, …)`	il nodo; nel padre: `last = T→sx`?	**attenzione** se entrambi i figli sono Nil sono uguali: la chiamata destra su Nil si simula "in linea" (vedi §4.6)
`A(T→sx, a, T→key)` e `A(T→dx, T→key, b)`	b; nel padre: `last = T→key`?	**no** se `T→key = b` le due chiamate hanno lo stesso b

4.4 Lo schema generale



Il ciclo alterna "scendo" (call = true) e "risalgo" (call = false).

SCHEMA (DUE CHIAMATE RICORSIVE)

```
AlgoIter(parametri)
  stack vuoti; call ← true; last ← ⊥
  while call or stack ≠ ø do
    if call then                    // FASE 1
      codice prima della chiamata 1
      if caso base then
        ret ← valore
        last ← parametro corrente
        call ← false
      else
        Push(variabili usate dopo)
        parametri ← quelli della chiam. 1
    else                            // risalita
      variabili ← Top(stack)
      if last = param. chiamata 1 then
        x ← ret                     // FASE 2
        codice tra le due chiamate
        Pop+Push dei valori aggiornati
        parametri ← quelli della chiam. 2
        call ← true
      else                          // FASE 3
        y ← ret
        codice dopo la chiamata 2
        ret ← valore restituito
        last ← parametro corrente
        Pop(tutti gli stack)
  return ret
```

"SIMULARE PRECISAMENTE"

Le funzioni esterne (G, H, ...) vanno chiamate **con gli stessi argomenti e nello stesso ordine** della versione ricorsiva (possono avere effetti collaterali). Anche i casi base vanno "eseguiti" (es. la chiamata su Nil che calcola `G(i)`).

COME LEGGERE LO SCHEMA

- Il ramo `if call` è la **discesa**: o è un caso base (restituisco e inizio a risalire) o salvo il contesto e scendo nella chiamata 1.
- Il ramo `else` è la **risalita**: ripristino il padre con i Top e uso `last` per sapere in quale fase riprendere.
- Se ci sono "return anticipati" tra le due chiamate (es. `if l = -1 then return -1`), anche lì vanno impostati `ret`, `last` e fatti i Pop.

4.5 Esempio guidato: il massimo di un array (dagli appunti)

RICORSIVO

```
1  Max(A, p, r)
2    if p < r then
3      q ← ⌊(p + r)/2⌋
4      v ← Max(A, p, q)          // chiamata 1
5      k ← Max(A, q+1, r)        // chiamata 2
6      return max(v, k)
7    else
8      return A[p]
```

Analisi delle variabili:

- r e q servono **dopo** la chiamata 1 (per la chiamata 2) ⇒ stack `Sr`, `Sq`.
- v è calcolato dopo la chiamata 1 e serve dopo la chiamata 2 ⇒ stack `Sv` (Push nella Fase 2).
- p non serve dopo: la chiamata 2 usa $q+1$.
- **last** = estremo destro della chiamata finita: la chiamata 1 finisce in $q < r$, la 2 in r.

ITERATIVO

```
1  MaxIter(A, p, r)
2    Sr, Sq, Sv ← ∅;  call ← true
3    while call or Sr ≠ ∅ do
4      if call then
5        if p < r then
6          q ← ⌊(p + r)/2⌋
7          Push(Sr, r); Push(Sq, q)
8          r ← q                          // chiamata 1 su (p, q)
9        else
10         ret ← A[p];  last ← r;  call ← false
11     else
12       r ← Top(Sr);  q ← Top(Sq)
13       if last ≠ r then                 // tornato dalla chiamata 1
14         v ← ret
15         Push(Sv, v)
16         p ← q + 1;  call ← true        // chiamata 2 su (q+1, r)
17       else                             // tornato dalla chiamata 2
18         k ← ret;  v ← Top(Sv)
19         ret ← max(v, k);  last ← r
20         Pop(Sr); Pop(Sq); Pop(Sv)
21   return ret
```

4.6 Esame 16/06/2026 — `Algoritmo(T, i)`

TESTO (G E H FUNZIONI ESTERNE)

```
function Algoritmo(T, i)
  a ← G(i)
  if T = Nil then
    return a
  else
    z ← Algoritmo(T→sx, 3·i)
    a ← H(z) + (T→key)·i + a
    z ← a + Algoritmo(T→dx, 3·i+2)
    return H(3·z)
```

Analisi:

Variabile	Serve dopo una chiamata?	Stack
T	sì: dopo la 1 servono `T→key` e `T→dx`	`ST`
i	sì: dopo la 1 (`·i`, `3·i+2`); serve anche per **last**	`Si`
a	sì: calcolata prima della 1 e usata dopo; poi **aggiornata** e usata dopo la 2	`Sa` (Pop+Push)
z	no: si calcola sempre da `ret` subito dopo	—

last = i della chiamata che termina: la chiamata 1 ha $3i$, la 2 ha $3i+2$ ⇒ nel padre testo `last = 3·i`.

SOLUZIONE ITERATIVA

```
function AlgoritmoIter(T, i)
  ST, Si, Sa ← ∅                          // uno stack per variabile
  call ← true;  last ← ⊥;  ret ← 0
  while call or ST ≠ ∅ do
    if call then                          // FASE 1: entro in una chiamata
      a ← G(i)
      if T = Nil then                     // caso base: return a
        ret ← a;  last ← i;  call ← false
      else                                // preparo la chiamata 1
        Push(ST, T); Push(Si, i); Push(Sa, a)
        T ← T→sx;  i ← 3·i
    else                                  // risalita: ripristino il padre
      T ← Top(ST);  i ← Top(Si);  a ← Top(Sa)
      if last = 3·i then                  // FASE 2: è finita la chiamata 1
        z ← ret
        a ← H(z) + (T→key)·i + a
        Pop(Sa); Push(Sa, a)              // aggiorno a salvato (serve dopo la 2)
        T ← T→dx;  i ← 3·i + 2;  call ← true
      else                                // FASE 3: è finita la chiamata 2
        z ← a + ret
        ret ← H(3·z);  last ← i
        Pop(ST); Pop(Si); Pop(Sa)
  return ret
```

VEDIAMOLA GIRARE: RADICE R (CHIAVE 5) CON SOLO FIGLIO SINISTRO B (CHIAVE 2), CHIAMATA CON $i = 1$

#	cosa fa il ciclo	S_T	S_i	S_a	last	call dopo
1	**F1** T=R, i=1: a ← G(1); Push(T,i,a); scendo a sx con i ← 3	R	1	a_R	⊥	true
2	**F1** T=B, i=3: a ← G(3); Push(T,i,a); scendo a sx con i ← 9	R B	1 3	a_R a_B	⊥	true
3	**F1** T=Nil, i=9: caso base → ret ← G(9), last ← 9	R B	1 3	a_R a_B	9	false
4	**F2** Top → T=B, i=3; last=9 = 3·3 ⇒ tornato dalla 1ª: a ← H(ret)+key·i+a; Pop+Push(Sa); scendo a dx con i ← 11	R B	1 3	a_R a'_B	9	true
5	**F1** T=Nil, i=11: caso base → ret ← G(11), last ← 11	R B	1 3	a_R a'_B	11	false
6	**F3** Top → T=B, i=3; last=11 ≠ 9 ⇒ tornato dalla 2ª: ret ← H(3(a+ret)); last ← 3; Pop di tutto	R	1	a_R	3	false
7	**F2** Top → T=R, i=1; last=3 = 3·1 ⇒ tornato dalla 1ª: a ← H(ret)+key·i+a; Pop+Push(Sa); scendo a dx con i ← 5	R	1	a'_R	3	true
8	**F1** T=Nil, i=5: caso base → ret ← G(5), last ← 5	R	1	a'_R	5	false
9	**F3** Top → T=R, i=1; last=5 ≠ 3 ⇒ tornato dalla 2ª: ret ← H(3(a+ret)); last ← 1; Pop di tutto	ø	ø	ø	1	false

Ordine delle funzioni esterne: G(1), G(3), G(9), H, G(11), H, H, G(5), H — identico alla versione ricorsiva. (Verificato al computer su 20 000 alberi casuali.)

4.7 Esame 14/07/2026 — `Algoritmo(T, a, b)`

TESTO

```
1  function Algoritmo(T, a, b)
2    if T = Nil then return 0
3    if T→key < a or T→key > b then
4      return -1
5    l ← Algoritmo(T→sx, a, T→key)
6    if l = -1 then return -1
7    r ← Algoritmo(T→dx, T→key, b)
8    if r = -1 or |l - r| > 1 then
9      return -1
10   return 1 + l + r
```

Nella foto della traccia le chiamate sono scritte su `T`: vanno lette come `T→sx` e `T→dx` (altrimenti la ricorsione non terminerebbe). Cosa calcola: controlla che le chiavi stiano negli intervalli (BST) e che i sottoalberi abbiano dimensioni che differiscono al più di 1; restituisce il numero di nodi o −1.

Variabile	Analisi	Stack
T	dopo la 1 servono `T→key`, `T→dx`	`ST`
b	la chiamata 1 lo cambia in `T→key`, ma la 2 usa il b originale	`Sb`
l	calcolata dopo la 1, usata dopo la 2	`Sl`
a, r	a non serve dopo la 1 (la 2 usa `T→key`); r si usa subito	—

LA SCELTA DI LAST (IL PUNTO DELICATO)

Le chiamate differiscono nel nodo (`T→sx` / `T→dx`) e negli estremi. Usare `last ← b` e testare `last = T→key` **non funziona** quando `T→key = b` (le due chiamate hanno lo stesso b: il padre crederebbe di tornare sempre dalla chiamata 1 e andrebbe in ciclo). Usiamo quindi **`last` = nodo della chiamata che termina** e testiamo `last = T→sx`; l'unico caso ambiguo (entrambi i figli Nil) si evita come nello schema del corso: se `T→dx = Nil` la chiamata 2 (che restituirebbe 0) si **simula in linea**, senza scendere.

SOLUZIONE ITERATIVA

```
function AlgoritmoIter(T, a, b)
  ST, Sb, Sl ← ∅
  call ← true;  last ← ⊥;  ret ← 0
  while call or ST ≠ ∅ do
    if call then                                        // FASE 1
      if T = Nil then                                   // return 0
        ret ← 0;  last ← T;  call ← false
      else if T→key < a or T→key > b then               // return -1
        ret ← -1;  last ← T;  call ← false
      else                                              // chiamata 1: (T→sx, a, T→key)
        Push(ST, T); Push(Sb, b)
        b ← T→key;  T ← T→sx
    else                                                // risalita
      T ← Top(ST);  b ← Top(Sb)
      if last = T→sx then                               // FASE 2: finita la chiamata 1
        l ← ret
        if l = -1 then                                  // return -1
          ret ← -1;  last ← T
          Pop(ST); Pop(Sb)
        else if T→dx ≠ Nil then                         // chiamata 2: (T→dx, T→key, b)
          Push(Sl, l)
          a ← T→key;  T ← T→dx;  call ← true
        else                                            // chiamata 2 su Nil: vale 0
          r ← 0
          if r = -1 or |l - r| > 1 then ret ← -1 else ret ← 1 + l + r
          last ← T
          Pop(ST); Pop(Sb)
      else                                              // FASE 3: finita la chiamata 2
        r ← ret;  l ← Top(Sl)
        if r = -1 or |l - r| > 1 then ret ← -1 else ret ← 1 + l + r
        last ← T
        Pop(ST); Pop(Sb); Pop(Sl)
  return ret
```

VERIFICA

Questa versione è stata confrontata al computer con la ricorsiva su 20 000 alberi casuali (anche con chiavi ripetute): stessi risultati. La variante con `last ← b`, invece, sbaglia o va in ciclo in molti dei casi di prova: tutti quelli in cui una chiave coincide con l'estremo `b`.

CHECKLIST DELL'ESERCIZIO 3

- Per ogni variabile: esiste prima di una chiamata? è letta dopo? è modificata dalla chiamata? ⇒ stack.
- **last** deve distinguere **sempre** le due chiamate: prova mentalmente il caso "figli entrambi Nil" e i casi limite (chiavi uguali agli estremi).
- **Top** all'inizio della risalita; **Pop** solo quando la chiamata del padre è davvero finita (anche nei return anticipati come `if l = -1 then return -1`!).
- Ogni ramo che "restituisce" deve impostare **ret**, **last** e (in Fase 1) **call ← false**.
- Il ciclo è `while call or stack ≠ ∅` e alla fine si restituisce **ret**.

ESERCIZIO 4 DELLO SCRITTO

5 Programmazione dinamica

C'è sempre. Ti chiedono di "descrivere l'approccio di PD" per un problema visto a lezione e di simularlo su un input: è un esercizio di **memoria + metodo**. Impara lo schema, poi le schede dei problemi.

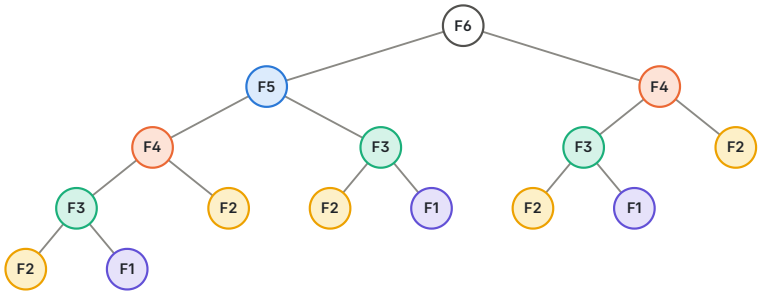
COSA TI CHIEDONO (SEMPRE QUESTI PUNTI)

- la **decomposizione** del problema in sottoproblemi;
- la **relazione ricorsiva di costo**;
- l'**algoritmo iterativo** e il suo **costo in tempo e spazio**;
- la **simulazione** su un input (riempire la tabella / matrice dei costi).

PROBLEMI VISTI A LEZIONE

- Fibonacci · taglio del tronco · intervalli pesati · LCS
- **albero binario di ricerca ottimo** (esame 14/07) · zaino 0/1
- **cammini minimi da sorgente singola** (esame 16/06) · tutte le coppie · Floyd-Warshall · cammino massimo su DAG

5.1 Perché serve: i sottoproblemi che si ripetono



Chiamate di Fib(6): stesso colore = stesso sottoproblema, ricalcolato più volte.

Nel divide et impera (Merge Sort) i sottoproblemi sono **disgiunti**. In Fibonacci, $Fib(n) = Fib(n-1) + Fib(n-2)$, invece, gli stessi sottoproblemi compaiono in rami diversi: il tempo soddisfa $T(n) = T(n-1) + T(n-2) + \Theta(1)$ e cresce come φ^n ($\varphi \approx 1.618$): **esponenziale**.

Ma i sottoproblemi **distinti** sono solo n. **Se i sottoproblemi distinti sono in numero polinomiale, li risolvo una volta sola e salvo il risultato in una tabella.** Questa è la programmazione dinamica.

RICORSIVO: Θ(Φⁿ)

```
Fib(n)
  if n ≤ 2 then return 1
  x ← Fib(n - 1)
  y ← Fib(n - 2)
  return x + y
```

MEMOIZZAZIONE (TOP-DOWN)

```
FibMemo(n, M)     // M[i] = -1:
  if M[n] = -1 then   // da fare
    x ← FibMemo(n - 1, M)
    y ← FibMemo(n - 2, M)
    M[n] ← x + y
  return M[n]
```

BOTTOM-UP: Θ(N)

```
FibIter(n)
  M[1] ← 1;  M[2] ← 1
  for i ← 3 to n do
    M[i] ← M[i-1] + M[i-2]
  return M[n]
```

5.2 Il metodo della PD (e lo schema di risposta)

1 · Formalizza	2 · Decomponi	3 · Teoremi	4 · Ricorrenza	5 · Algoritmo
input, output, soluzione ammissibile, costo	una scelta + un sottoproblema più piccolo	Th1, Th2 (ammissibilità), Th3 (ottimalità)	costo ottimo = min/max sulle scelte	tabella, ordine, costo, ricostruzione

I TRE TEOREMI (SEMPRE LO STESSO SCHEMA)

- **Th 1 (decomposizione):** se S è ammissibile per P, togliendo la scelta resta una soluzione **ammissibile** del sottoproblema.
- **Th 2 (composizione):** se S' è ammissibile per il sottoproblema, aggiungendo la scelta ottengo una soluzione **ammissibile** di P.
- **Th 3 (sottostruttura ottima):** se S è **ottima**, la parte che risolve il sottoproblema è ottima. *Dim. per assurdo* ("taglia e incolla"): se esistesse S' migliore per il sottoproblema, per Th 2 sostituendola otterrei una soluzione di P migliore di S: assurdo.

IL COSTO, SEMPRE COSÌ

Tempo = (numero di sottoproblemi distinti) × (numero di scelte per sottoproblema) × (costo di una scelta).

Spazio = dimensione della tabella (+ tabella delle scelte S per ricostruire la soluzione).

Ordine di riempimento: guarda da quali celle dipende $C[\cdot]$: devono essere calcolate **prima**.

SCHEMA DI RISPOSTA PER L'ESERCIZIO 4

1. **Problema e soluzioni**: input, output, quando una soluzione è ammissibile, funzione costo.
2. **Decomposizione**: quale scelta faccio, quale sottoproblema resta; i sottoproblemi sono $P(\dots)$ con parametri...; quanti sono.
3. **Relazione ricorsiva** con casi base (giustificata dai tre teoremi, enunciati in breve).
4. **Algoritmo iterativo** (bottom-up) + funzione che ricostruisce la soluzione; **costo in tempo e spazio**.
5. **Simulazione**: tabella/e riempite nell'ordine dell'algoritmo, risultato finale e soluzione ricostruita.

5.3 Taglio del tronco

Problema	Tronco di lunghezza L; un pezzo di lunghezza i vale $V(i)$. Tagliarlo (lunghezze intere) massimizzando il valore totale. Soluzione: $(l_1, \ldots, l_k)$ con $\sum l_i = L$. Le decomposizioni possibili sono 2^{L-1}.
Scelta	la lunghezza y del **primo pezzo**; resta $P(L-y)$. Sottoproblemi: $P(0), \ldots, P(L) \Rightarrow L+1$.
Ricorrenza	$C(0) = 0, \qquad C(x) = \max_{1 \le y \le x}\{V(y) + C(x-y)\}$
Costo	ricorsivo diretto: $T(n) = 1 + \sum_{j<n} T(j) = 2^n$; bottom-up: $\Theta(L^2)$ **tempo,** $\Theta(L)$ **spazio**.

BOTTOM-UP + RICOSTRUZIONE

```
1  CostoOttimo(L, V)
2    C[0] ← 0
3    for l ← 1 to L do
4      v ← -∞
5      for y ← 1 to l do
6        if v < V[y] + C[l - y] then
7          v ← V[y] + C[l - y];  t ← y
8      C[l] ← v;  S[l] ← t
9    return C, S
10
11 StampaSol(L, S)
12   l ← L
13   while l > 0 do
14     print S[l];  l ← l - S[l]
```

ESEMPIO: $L = 8$

lunghezza x	0	1	2	3	4	5	6	7	8
V(x) prezzo	–	1	5	8	9	10	17	17	20
C[x] ottimo	0	1	5	8	10	13	17	18	22
S[x] 1° pezzo	–	1	2	3	2	2	6	1	2

Risultato: C[8] = 22, pezzi: 2 + 6 (cioè $V(2) + V(6) = 5 + 17$).

5.4 Schedulazione di intervalli pesati

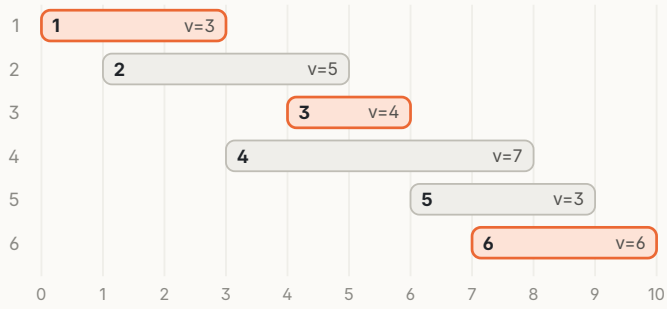
Problema	n attività $[s(i), e(i))$ con valore $v(i)$; scegliere attività a due a due **compatibili** ($e(i) \le s(j)$ **oppure** $e(j) \le s(i)$) di valore totale massimo.
Preparazione	ordino per tempo di **fine**; $l(j)$ = massimo indice $i < j$ con $e(i) \le s(j)$ (0 se non esiste): le attività $1..l(j)$ sono tutte compatibili con j. Si calcola con ricerca binaria.
Scelta (binaria)	l'attività j è nella soluzione ⇒ resta $P(l(j))$; non c'è ⇒ resta $P(j-1)$. (Scegliere "quale è l'ultima attività" tra n possibilità darebbe $\Theta(n^2)$.)
Ricorrenza	$C(0) = 0, \qquad C(j) = \max\{\, C(j-1),\ v(j) + C(l(j)) \,\}$
Costo	$\Theta(n)$ per la tabella; ordinamento + ricerche binarie $\Theta(n \log n) \Rightarrow \Theta(n \log n)$; spazio $\Theta(n)$.

ALGORITMO

```
CostoOttimo(n, v, l)
  C[0] ← 0
  for j ← 1 to n do
    C[j] ← C[j-1]
    if v[j] + C[l[j]] > C[j] then
      C[j] ← v[j] + C[l[j]]
  return C

Sol(i, C, l)          // ricostruzione
  if i > 0 then
    if C[i] = C[i-1] then
      Sol(i-1, C, l)
    else
      Sol(l[i], C, l);  print i
```

ESEMPIO (ATTIVITÀ GIÀ ORDINATE PER FINE)



j	0	1	2	3	4	5	6
[s,e)	–	[0,3)	[1,5)	[4,6)	[3,8)	[6,9)	[7,10)
v(j)	–	3	5	4	7	3	6
l(j)	–	0	0	1	1	3	3
C[j]	**0**	**3**	**5**	**7**	**10**	**10**	**13**

Ricostruzione: soluzione {1, 3, 6} con valore 13 (in arancione).

5.5 Sottosequenza comune più lunga (LCS)

Problema	$X = x_1 \dots x_n$, $Y = y_1 \dots y_m$: la più lunga Z sottosequenza (non necessariamente contigua) di entrambe. Soluzione = **matching** $(i_1, j_1), \dots, (i_k, j_k)$ con $x_{i_l} = y_{j_l}$ e indici strettamente crescenti in entrambe.
Sottoproblemi	$P(i,j)$ = LCS dei prefissi $x_1..x_i$ e $y_1..y_j$ ⇒ $(n+1)(m+1)$. Guardo gli **ultimi caratteri**.
Ricorrenza	$C(i,j) = 0$ se $i = 0$ o $j = 0$; $C(i,j) = 1 + C(i-1, j-1)$ se $x_i = y_j$; $C(i,j) = \max\{C(i-1,j), C(i,j-1)\}$ se $x_i \neq y_j$.
Perché	se $x_i = y_j$ conviene sempre accoppiarli (aggiungere un carattere allunga la LCS al più di 1); se sono diversi almeno uno dei due non è usato.
Costo	tabella per righe: $\Theta(nm)$ **tempo e spazio**; ricostruzione $\Theta(n+m)$.

ALGORITMO

```
1  LCS(X, Y, n, m)
2    for i ← 0 to n do C[i,0] ← 0
3    for j ← 0 to m do C[0,j] ← 0
4    for i ← 1 to n do
5      for j ← 1 to m do
6        if X[i] = Y[j] then
7          C[i,j] ← 1 + C[i-1,j-1]
8        else if C[i-1,j] < C[i,j-1] then
9          C[i,j] ← C[i,j-1]
10       else
11         C[i,j] ← C[i-1,j]
12   return C
13
14 PrintSol(i, j)
15   if i > 0 and j > 0 then
16     if X[i] = Y[j] then
17       PrintSol(i-1, j-1); print X[i]
18     else if C[i,j] = C[i-1,j] then
19       PrintSol(i-1, j)
20     else PrintSol(i, j-1)
```

ESEMPIO DAGLI APPUNTI: $X = ABCDABC$, $Y = BDACB$

		B_1	D_2	A_3	C_4	B_5
	0	0	0	0	0	0
A_1	0	↑ 0	↑ 0	↖ 1	← 1	← 1
B_2	0	↖ 1	← 1	↑ 1	↑ 1	↖ 2
C_3	0	↑ 1	↑ 1	↑ 1	↖ 2	↑ 2
D_4	0	↑ 1	↖ 2	← 2	↑ 2	↑ 2
A_5	0	↑ 1	↑ 2	↖ 3	← 3	← 3
B_6	0	↖ 1	↑ 2	↑ 3	↑ 3	↖ 4
C_7	0	↑ 1	↑ 2	↑ 3	↖ 4	↑ 4

Freccia: ↖ match (diagonale), ↑ vengo da sopra, ← da sinistra. In giallo il cammino di ricostruzione (bordo = carattere preso).

Risultato: lunghezza 4, una LCS: **BDAB**.

5.6 Segmentazione di un testo integrazione

(Dalle dispense dell'altro corso.) Dato un testo $T[1..m]$ e una funzione `Dict(T,i,j)` che dice se $T[i..j]$ è una parola del dizionario, decidere se T si spezza in parole. Sottoproblema $P(i)$: il suffisso $T[i..m]$ è segmentabile? Scelta: la prima parola $T[i..j]$.

$Seg(m+1) = \text{true}; \quad Seg(i) = \bigvee_{j=i}^{m} \big(Dict(T,i,j) \wedge Seg(j+1)\big)$

Senza memoria $T(m) \geq 2^m$; con la tabella $m+1$ sottoproblemi × $O(m)$ scelte ⇒ $\Theta(m^2)$.

BOTTOM-UP

```
1  Segmentabile(T, m)
2    Seg[m+1] ← true
3    for i ← m downto 1 do
4      Seg[i] ← false
5      for j ← i to m do
6        if Dict(T, i, j) and Seg[j+1] then
7          Seg[i] ← true
8    return Seg[1]
```

5.7 Albero binario di ricerca ottimo

Problema	Chiavi ordinate $k_1 < \cdots < k_n$ (vettore A) con frequenze/probabilità di accesso $F[1..n]$. Costruire il BST che contiene esattamente queste chiavi e ha **costo medio di ricerca minimo**: $C(T) = \sum_i F[i] \cdot (\text{prof}_T(i) + 1)$ (radice a profondità 0). Gli alberi possibili sono $\Theta(4^n/n^{3/2})$: troppi da provare.
Decomposizione	Scelgo la **radice** k tra $i..j$: il sottoalbero sinistro contiene esattamente $i..k-1$, il destro $k+1..j$ (proprietà del BST). Sottoproblemi $P(i,j)$ = albero ottimo per le chiavi $i..j$ ⇒ $\Theta(n^2)$; scelte per sottoproblema: $j-i+1$.
Il costo si spezza	Appendendo T' e T'' sotto la radice, ogni loro nodo scende di un livello: $C(T) = \sum_{z=i}^{j} F[z] + C(T') + C(T'')$.
Ricorrenza	$C(i,j) = 0$ se $i > j$; $\quad C(i,j) = \sum_{z=i}^{j} F[z] + \min_{i \le k \le j} \{ C(i,k-1) + C(k+1,j) \}$ se $i \le j$.
Teoremi	**Th1**: se T con radice k è ammissibile per $P(i,j)$, allora T_{sx} lo è per $P(i,k-1)$ e T_{dx} per $P(k+1,j)$. **Th2**: viceversa, una radice k con due BST ammissibili per $P(i,k-1)$ e $P(k+1,j)$ forma un BST ammissibile per $P(i,j)$. **Th3**: se T è ottimo lo sono T_{sx} e T_{dx} (per assurdo: sostituendo un T' più economico, per la formula del costo C scenderebbe).
Costo	$\Theta(n^2)$ sottoproblemi × $O(n)$ scelte = $\Theta(n^3)$ **tempo**, $\Theta(n^2)$ **spazio** (matrici C e S).

ALGORITMO ITERATIVO

```
1  ABROttimo(n, F)
2    for i ← 1 to n+1 do C[i, i-1] ← 0
3    for i ← n downto 1 do
4      W ← 0
5      for j ← i to n do
6        W ← W + F[j]          // F[i]+…+F[j]
7        C[i,j] ← +∞
8        for k ← i to j do
9          q ← W + C[i,k-1] + C[k+1,j]
10         if q < C[i,j] then
11           C[i,j] ← q;  S[i,j] ← k
12   return C, S
```

RICOSTRUZIONE DELL'ALBERO

```
1  BuildTree(S, i, j)
2    if i > j then return Nil
3    T ← CreaNodo(A[S[i,j]])
4    T→sx ← BuildTree(S, i, S[i,j] - 1)
5    T→dx ← BuildTree(S, S[i,j] + 1, j)
6    return T
```

ORDINE DI RIEMPIMENTO

$C[i,j]$ usa $C[i,k-1]$ (stessa riga, più a sinistra) e $C[k+1,j]$ (stessa colonna, più in basso) ⇒ righe **dal basso** ($i = n..1$), e in ogni riga j crescente. Si riempie solo il triangolo $i \le j$.

Esame 14/07/2026 · Es. 4

Simulare su $A = [1,2,3,4,5]$ e $F = [\frac{4}{19}, \frac{8}{19}, \frac{2}{19}, \frac{1}{19}, \frac{4}{19}]$.

Trucco di calcolo: tutte le frequenze hanno denominatore 19 ⇒ lavoro sui **numeratori** $F = [4,8,2,1,4]$ e divido per 19 solo alla fine. $W(i,j) = F[i] + \cdots + F[j]$.

Calcolo cella per cella, nell'ordine dell'algoritmo ($i = 5, \ldots, 1$; j crescente). Per ogni radice k si somma $C(i,k-1) + C(k+1,j)$ e si prende il minimo (in arancione):

cella	W(i,j)	candidati: C(i,k−1)+C(k+1,j) per ogni radice k	C = W + min	S (radice)
C(5,5)	4	**k=5: 0+0**	4	5
C(4,4)	1	**k=4: 0+0**	1	4
C(4,5)	5	k=4: 0+4 **k=5: 1+0**	6	5
C(3,3)	2	**k=3: 0+0**	2	3
C(3,4)	3	**k=3: 0+1** k=4: 2+0	4	3
C(3,5)	7	k=3: 0+6 k=4: 2+4 **k=5: 4+0**	11	5
C(2,2)	8	**k=2: 0+0**	8	2
C(2,3)	10	**k=2: 0+2** k=3: 8+0	12	2
C(2,4)	11	**k=2: 0+4** k=3: 8+1 k=4: 12+0	15	2
C(2,5)	15	**k=2: 0+11** k=3: 8+6 k=4: 12+4 k=5: 15+0	26	2
C(1,1)	4	**k=1: 0+0**	4	1
C(1,2)	12	k=1: 0+8 **k=2: 4+0**	16	2
C(1,3)	14	k=1: 0+12 **k=2: 4+2** k=3: 16+0	20	2
C(1,4)	15	k=1: 0+15 **k=2: 4+4** k=3: 16+1 k=4: 20+0	23	2
C(1,5)	19	k=1: 0+26 **k=2: 4+11** k=3: 16+6 k=4: 20+4 k=5: 23+0	34	2

$W(i,j)$ **(×1/19)**

i \ j	0	1	2	3	4	5
1	–	4	12	14	15	19
2		–	8	10	11	15
3			–	2	3	7
4				–	1	5
5					–	4
6						–

$C(i,j)$ **(×1/19)**

i \ j	0	1	2	3	4	5
1	0	4	16	20	23	34
2		0	8	12	15	26
3			0	2	4	11
4				0	1	6
5					0	4
6						0

$S(i,j)$ **= radice**

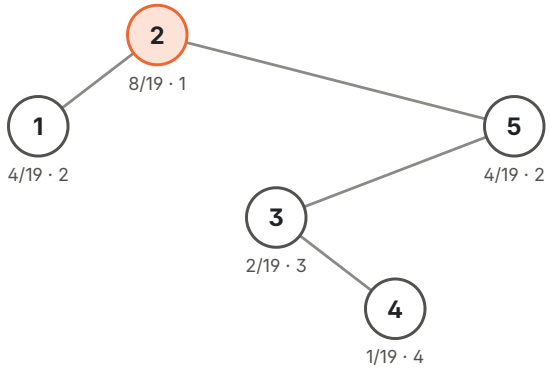
i \ j	0	1	2	3	4	5
1	–	1	2	2	2	2
2		–	2	2	2	2
3			–	3	3	5
4				–	4	5
5					–	5
6						–

Ricostruzione con BuildTree: $S(1,5)=2$ ⇒ radice **2**; a sinistra $S(1,1)=1$; a destra $S(3,5)=5$ ⇒ nodo 5, con sinistro $S(3,4)=3$, il cui destro è $S(4,4)=4$.

RISULTATO

Costo minimo $C(1,5)=\dfrac{34}{19}\approx 1.79$. Controllo: $8\cdot 1+4\cdot 2+4\cdot 2+2\cdot 3+1\cdot 4=34$ ✔

Per confronto, l'albero perfettamente bilanciato (radice 3) costa $\frac{44}{19}\approx 2.32$: **l'ottimo non è il bilanciato**.



Albero ottimo (sotto ogni nodo: frequenza · n° confronti).

5.8 Zaino 0/1

Problema	n oggetti con peso $P(i)$ e valore $v(i)$, capacità K. Scegliere $X \subseteq \{1..n\}$ con $\sum_X P \leq K$ e valore massimo. Soluzione: indici $i_1 < \cdots < i_k$.
Scelta (binaria)	l'oggetto i è nello zaino ⇒ resta $P(i-1,\, j-P(i))$; non c'è ⇒ resta $P(i-1,j)$. Sottoproblemi $P(i,j)$: primi i oggetti, capacità j ⇒ $(n+1)(K+1)$. (Scegliere "l'ultimo oggetto preso" tra n darebbe $O(n^2K)$.)
Ricorrenza	$C(i,j)=0$ se $i=0$ o $j=0$; $C(i,j)=C(i-1,j)$ se $P(i)>j$; $C(i,j)=\max\{C(i-1,j),\, v(i)+C(i-1,j-P(i))\}$ altrimenti.
Costo	$\Theta(nK)$ tempo e spazio — ma è **pseudopolinomiale**: K occupa $\log K$ bit, quindi $nK = n\,2^{\log K}$ è esponenziale nella dimensione dell'input (il problema è NP-difficile).

ALGORITMO

```
1  Zaino(n, K, P, v)
2    for j ← 0 to K do C[0,j] ← 0
3    for i ← 1 to n do
4      C[i,0] ← 0
5      for j ← 1 to K do
6        C[i,j] ← C[i-1,j]
7        if P[i] ≤ j then
8          q ← v[i] + C[i-1, j-P[i]]
9          if q > C[i,j] then C[i,j] ← q
10   return C[n,K]
```

ESEMPIO: $K = 5$

i \ j	0	1	2	3	4	5	oggetto i
0	0	0	0	0	0	0	–
1	0	0	3	3	3	3	P=2, v=3
2	0	0	3	4	4	7	P=3, v=4
3	0	0	3	4	5	7	P=4, v=5
4	0	0	3	4	5	7	P=5, v=6

Ricostruzione (giallo): se $C[i,j] = C[i-1,j]$ l'oggetto i non è preso, altrimenti lo prendo e vado a $(i-1, j-P[i])$. Risultato: oggetti {1, 2}, valore 7.

5.9 Cammini minimi da sorgente singola (Bellman-Ford come PD)

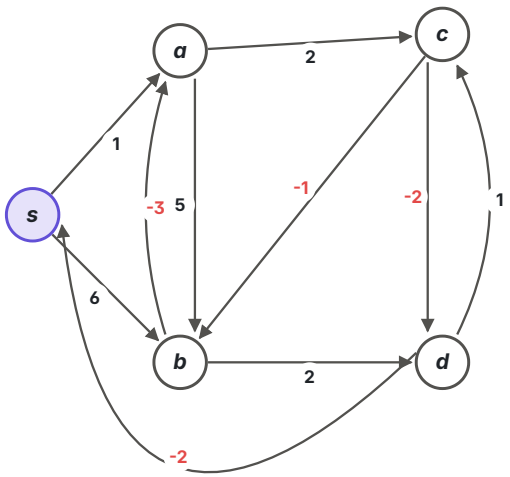
Problema	Grafo orientato pesato $G = \langle V, E, w \rangle$, $w : E \to \mathbb{R}$ (pesi anche negativi), **senza cicli di peso negativo**, sorgente s. Per ogni v: il cammino $s = v_0 \to \cdots \to v_k = v$ di peso $\sum w(v_{i-1}, v_i)$ minimo.
Il problema della decomposizione "naturale"	Togliendo l'ultimo arco (z, v): $C(v) = \min_{(z,v)\in E}\{C(z) + w(z,v)\}$. Con i cicli è **mal definita**: $C(t)$ dipende da $C(r)$ che dipende da $C(t)$... non esiste un ordine di calcolo.
Soluzione: un parametro che scende	$P(v,l)$ = cammino minimo da s a v con **al più l archi**. Senza cicli negativi un cammino minimo può essere scelto **semplice** ⇒ al più $\lvert V\rvert - 1$ archi: il problema principale è $P(v, \lvert V\rvert - 1)$. Sottoproblemi: $\lvert V\rvert \cdot \lvert V\rvert$.
Ricorrenza	$C(s,l) = 0$; $\quad C(v,0) = \infty$ per $v \neq s$; $\quad C(v,l) = \min\left\{C(v,l-1),\ \min_{(x,v)\in E}\{w(x,v) + C(x,l-1)\}\right\}$
Teoremi	**Th1**: se π (con $\leq l$ archi) termina con l'arco (x,v), il prefisso è un cammino $s \rightsquigarrow x$ con $\leq l-1$ archi. **Th2**: un cammino $s \rightsquigarrow x$ con $\leq l-1$ archi più l'arco (x,v) è un cammino $s \rightsquigarrow v$ con $\leq l$ archi. **Th3**: il prefisso di un cammino ottimo è ottimo (per assurdo).
Costo	per ogni l si esamina ogni arco una volta: $\Theta(\lvert V\rvert \cdot \lvert E\rvert)$ tempo (+$\Theta(\lvert V\rvert^2)$ per la tabella); spazio $\Theta(\lvert V\rvert^2)$ (basterebbero due colonne: $\Theta(\lvert V\rvert)$). È la complessità di Bellman-Ford; l'istruzione interna è proprio il **rilassamento** dell'arco (x,v).

ALGORITMO ITERATIVO (MATRICE DEI COSTI C E DELLE SCELTE S)

```
1  CamminiMinimi(G, w, s)                          // n = |V|
2    for each v ∈ V do C[v,0] ← ∞;  S[v,0] ← ⊥
3    C[s,0] ← 0
4    for l ← 1 to n - 1 do                         // colonne da sinistra a destra
5      C[s,l] ← 0
6      for each v ∈ V \ {s} do
7        C[v,l] ← C[v,l-1];  S[v,l] ← S[v,l-1]     // caso "al più l-1 archi"
8        for each x tale che (x,v) ∈ E do          // ultimo arco (x,v)
9          if w(x,v) + C[x,l-1] < C[v,l] then      // = relax(x, v)
10           C[v,l] ← w(x,v) + C[x,l-1];  S[v,l] ← x
11   return C, S
12
13 Percorso(S, v, l)                               // stampa il cammino s ⇝ v
14   if v = s then print s
15   else Percorso(S, S[v,l], l - 1);  print v
```

Variante (dispense dell'altro corso): fissando la **destinazione** t e usando i **successori** invece dei predecessori si ottengono i cammini minimi da tutti i vertici verso t; la tabella è identica come struttura.

Esame 16/06/2026 · Es. 4 Simulare l'algoritmo sul grafo dato compilando la matrice dei costi.



Il grafo della traccia (pesi negativi in rosso).

vertice	predecessori (arco, peso)
a	$(s,a)\,1 \cdot (b,a)\,-3$
b	$(s,b)\,6 \cdot (a,b)\,5 \cdot (c,b)\,-1$
c	$(a,c)\,2 \cdot (d,c)\,1$
d	$(b,d)\,2 \cdot (c,d)\,-2$
s	$(d,s)\,-2$ (ignorato: $C[s,l]=0$)

Matrice dei costi $C[v,l]$ (sotto: il predecessore $S[v,l]$). $|V|=5 \Rightarrow$ colonne $l=0..4$. In azzurro le celle che migliorano rispetto alla colonna precedente.

v \ l	l = 0	l = 1	l = 2	l = 3	l = 4
s	**0**	**0**	**0**	**0**	**0**
a	**∞** –	**1** s	**1** s	**1** s	**-1** b
b	**∞** –	**6** s	**6** s	**2** c	**2** c
c	**∞** –	**∞** –	**3** a	**3** a	**2** d
d	**∞** –	**∞** –	**8** b	**1** c	**1** c

Lettura della colonna finale ($l=4$) e cammini ricostruiti con `Percorso`:

- b: 2 con $s \to a \to c \to b$ $(1+2-1)$
- d: 1 con $s \to a \to c \to d$ $(1+2-2)$
- a: -1 con $s \to a \to c \to b \to a$ (passa due volte per a!)
- c: 2 con $s \to a \to c \to d \to c$ (passa due volte per c!)

CALCOLO DETTAGLIATO DI OGNI CELLA

cella	min tra: (resto con l−1 archi) e (ultimo arco (x,v) + C[x,l−1])	valore
C[a,1]	C[a,0] = ∞ · 1 + C[s,0] = 1 · -3 + C[b,0] = ∞	**1**
C[b,1]	C[b,0] = ∞ · 5 + C[a,0] = ∞ · 6 + C[s,0] = 6 · -1 + C[c,0] = ∞	**6**
C[c,1]	C[c,0] = ∞ · 2 + C[a,0] = ∞ · 1 + C[d,0] = ∞	**∞**
C[d,1]	C[d,0] = ∞ · -2 + C[c,0] = ∞ · 2 + C[b,0] = ∞	**∞**
C[a,2]	C[a,1] = 1 · 1 + C[s,1] = 1 · -3 + C[b,1] = 3	**1**
C[b,2]	C[b,1] = 6 · 5 + C[a,1] = 6 · 6 + C[s,1] = 6 · -1 + C[c,1] = ∞	**6**
C[c,2]	C[c,1] = ∞ · 2 + C[a,1] = 3 · 1 + C[d,1] = ∞	**3**
C[d,2]	C[d,1] = ∞ · -2 + C[c,1] = ∞ · 2 + C[b,1] = 8	**8**
C[a,3]	C[a,2] = 1 · 1 + C[s,2] = 1 · -3 + C[b,2] = 3	**1**
C[b,3]	C[b,2] = 6 · 5 + C[a,2] = 6 · 6 + C[s,2] = 6 · -1 + C[c,2] = 2	**2**
C[c,3]	C[c,2] = 3 · 2 + C[a,2] = 3 · 1 + C[d,2] = 9	**3**
C[d,3]	C[d,2] = 8 · -2 + C[c,2] = 1 · 2 + C[b,2] = 8	**1**
C[a,4]	C[a,3] = 1 · 1 + C[s,3] = 1 · -3 + C[b,3] = -1	**-1**
C[b,4]	C[b,3] = 2 · 5 + C[a,3] = 6 · 6 + C[s,3] = 6 · -1 + C[c,3] = 2	**2**
C[c,4]	C[c,3] = 3 · 2 + C[a,3] = 3 · 1 + C[d,3] = 2	**2**
C[d,4]	C[d,3] = 1 · -2 + C[c,3] = 1 · 2 + C[b,3] = 4	**1**

OCCHIO: QUESTO GRAFO HA CICLI DI PESO NEGATIVO

Il testo assume che non ce ne siano, ma leggendo il disegno: $a \to c \to b \to a$ pesa $2 - 1 - 3 = -2$, $c \to d \to c$ pesa $-2 + 1 = -1$, $s \to a \to c \to d \to s$ pesa $1 + 2 - 2 - 2 = -1$. Per questo nella colonna $l = 4$ compaiono cammini **non semplici** (per a e c), e un'ulteriore iterazione $l = 5$ migliorerebbe ancora ($a : -1$, $b : 1$, $c : 1$, $d : 0$): è esattamente il **test di Bellman-Ford** per rilevare i cicli negativi (se dopo $|V| - 1$ passi qualche arco si rilassa ancora, c'è un ciclo negativo).

All'esame: compila la matrice esattamente come sopra (l'algoritmo è meccanico e termina comunque), poi aggiungi una riga che segnala il ciclo negativo: i valori di $C[\cdot, |V| - 1]$ sono i costi minimi dei cammini con al più 4 archi, non distanze vere. Mostra che hai capito l'ipotesi.

5.10 Cammini minimi tra tutte le coppie

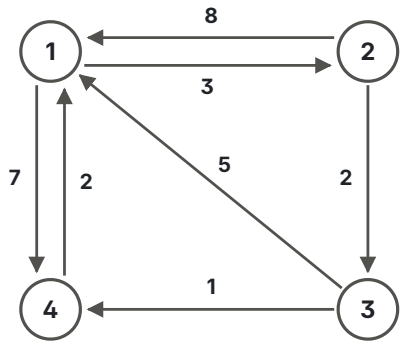
Stessa idea, ma con due estremi variabili: $C(i,j,\cdot)$. Tre soluzioni, sempre più veloci:

Soluzione	Sottoproblema	Ricorrenza	Costo
1 · Espansione	$P(i,j,k)$: $\le k$ archi	$C(i,j,k) = \min_{1\le m\le n}\{C(i,m,k-1) + w(m,j)\}$ (con $w(j,j)=0$) — un "prodotto di matrici" con $(\min,+)$ al posto di $(+,\times)$	$n-2$ espansioni da $\Theta(n^3)$: $\Theta(n^4)$
2 · Raddoppio	$\le 2^r$ archi	$C^{2a} = C^a \oplus C^a$: un cammino con $\le 2a$ archi si spezza in un vertice intermedio in due da $\le a$	$\lceil \log(n-1) \rceil$ espansioni: $\Theta(n^3 \log n)$
3 · Floyd-Warshall	$P(i,j,m)$: vertici **intermedi** solo in $\{1..m\}$	$C(i,j,0) = w(i,j)$ (0 se $i=j$, ∞ senza arco); $C(i,j,m) = \min\{C(i,j,m-1),\ C(i,m,m-1) + C(m,j,m-1)\}$	2 scelte per n^3 sottoproblemi: $\Theta(n^3)$

Perché Floyd-Warshall funziona. Sia π un cammino minimo semplice da i a j con intermedi in $\{1..m\}$. O $m \notin \pi$, e allora π risolve $P(i,j,m-1)$; oppure $m \in \pi$ **una volta sola**, e $\pi = \pi_1 \cdot \pi_2$ con $\pi_1 : i \rightsquigarrow m$ e $\pi_2 : m \rightsquigarrow j$ che usano intermedi solo in $\{1..m-1\}$.

FLOYD-WARSHALL

```
1  FloydWarshall(W, n)
2    for i ← 1 to n do
3      for j ← 1 to n do C[i,j,0] ← W[i,j]
4    for m ← 1 to n do
5      for i ← 1 to n do
6        for j ← 1 to n do
7          C[i,j,m] ← C[i,j,m-1]
8          q ← C[i,m,m-1] + C[m,j,m-1]
9          if q < C[i,j,m] then C[i,j,m] ← q
10
11 Percorso(i, j, m)
12   if m = 0 then print (i, j)
13   else if C[i,j,m] = C[i,j,m-1] then
14     Percorso(i, j, m-1)
15   else
16     Percorso(i, m, m-1)
17     Percorso(m, j, m-1)
```



Grafo d'esempio.

COME SI COMPILA A MANO

Al passo m la **riga m e la colonna m non cambiano** (viola). Per ogni altra cella: $C[i,j] \leftarrow \min(C[i,j],\ C[i,m] + C[m,j])$ — "conviene passare da m?". In arancione le celle migliorate.

$C^{(0)}$ = W

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$C^{(1)}$ · intermedi ⊆ {1..1}

	1	2	3	4
1	0	3	∞	7
2	8	0	2	**15**
3	5	**8**	0	1
4	2	**5**	∞	0

$C^{(2)}$ · intermedi ⊆ {1..2}

	1	2	3	4
1	0	3	**5**	7
2	8	0	2	15
3	5	8	0	1
4	2	5	**7**	0

$C^{(3)}$ · intermedi ⊆ {1..3}

	1	2	3	4
1	0	3	5	**6**
2	**7**	0	2	**3**
3	5	8	0	1
4	2	5	7	0

$C^{(4)}$ · intermedi ⊆ {1..4}

	1	2	3	4
1	0	3	5	6
2	**5**	0	2	3
3	**3**	**6**	0	1
4	2	5	7	0

5.11 Cammino semplice massimo: quando la PD non basta

Con i cicli un cammino "massimo" può non esistere (giro nel ciclo all'infinito): si chiede il cammino **semplice** più lungo da s a t. La decomposizione naturale $S(s,t) = s \to S(v_1,t)$ ha **Th 1** ma **non Th 2**: $S'(v_1,t) = v_1 \to s \to v_2 \to t$ è semplice, ma $s \to S'(v_1,t)$ ripete s.

Una decomposizione corretta deve ricordare i vertici già usati, $P(u,X)$ con $X \subseteq V$: i sottoproblemi sono $|V| \cdot 2^{|V|}$, **esponenziali**. Il problema è **NP-completo**.

SU UN DAG FUNZIONA

Se G è **aciclico**, $s \to S'(v_1,t)$ non può ripetere s (sarebbe un ciclo): vale Th 2. Ricorrenza: $C(t) = 0$, $C(v) = \max_{x \in Adj(v)}\{1 + C(x)\}$ ($-\infty$ se nessun successore raggiunge t). L'ordine non è ovvio (servirebbe un ordinamento topologico) ⇒ **memoizzazione**: ogni vertice e arco visti una volta, $\Theta(|V| + |E|)$.

5.12 Riepilogo della programmazione dinamica

Problema	Sottoproblema	Scelte	Ricorrenza (caso generale)	Tempo	Spazio
Taglio tronco	$C(x)$, lunghezza x	x	$\max_y\{V(y) + C(x-y)\}$	$\Theta(L^2)$	$\Theta(L)$
Intervalli pesati	$C(j)$, prime j	2	$\max\{C(j-1), v(j) + C(l(j))\}$	$\Theta(n \log n)$	$\Theta(n)$
LCS	$C(i,j)$, prefissi	1–2	$1 + C(i-1,j-1)$ o $\max\{C(i-1,j), C(i,j-1)\}$	$\Theta(nm)$	$\Theta(nm)$
ABR ottimo	$C(i,j)$, chiavi $i..j$	$j-i+1$	$W(i,j) + \min_k\{C(i,k-1) + C(k+1,j)\}$	$\Theta(n^3)$	$\Theta(n^2)$
Zaino 0/1	$C(i,j)$, oggetti, capacità	2	$\max\{C(i-1,j), v_i + C(i-1,j-P_i)\}$	$\Theta(nK)$*	$\Theta(nK)$
Sorgente singola	$C(v,l)$, $\leq l$ archi	in-deg(v) + 1	$\min\{C(v,l-1), \min_x\{w(x,v) + C(x,l-1)\}\}$	$\Theta(\|V\|\|E\|)$	$\Theta(\|V\|^2)$
Floyd-Warshall	$C(i,j,m)$, intermedi $\leq m$	2	$\min\{C(i,j,m-1), C(i,m,m-1) + C(m,j,m-1)\}$	$\Theta(n^3)$	$\Theta(n^3)$ ($\Theta(n^2)$)
Cammino max DAG	$C(v)$	out-deg(v)	$\max_x\{1 + C(x)\}$	$\Theta(\|V\| + \|E\|)$	$\Theta(\|V\|)$

* pseudopolinomiale.

ESERCIZIO 4 (ALTERNATIVA ALLA PD)

6 Algoritmi greedy

Quando una sola scelta "localmente migliore" porta sempre a una soluzione ottima, non serve provare tutte le scelte: si risolve **un solo** sottoproblema.

6.1 L'idea

Per un problema P la PD considera **tutte** le scelte $c_1, \dots, c_k$, risolve i sottoproblemi $P_1, \dots, P_k$ e prende il migliore. Un algoritmo **greedy** (goloso) fa **una sola** scelta, quella che sembra migliore adesso, e prosegue sull'unico sottoproblema rimasto.

Funziona se valgono **due** proprietà:

- **sottostruttura ottima** (Th 1–3, come nella PD);
- **proprietà della scelta greedy** (Th 4): esiste una soluzione ottima che contiene la scelta greedy. Si dimostra con un **argomento di scambio**: prendo una soluzione ottima qualsiasi e la modifico, senza peggiorarla, finché contiene la scelta greedy.

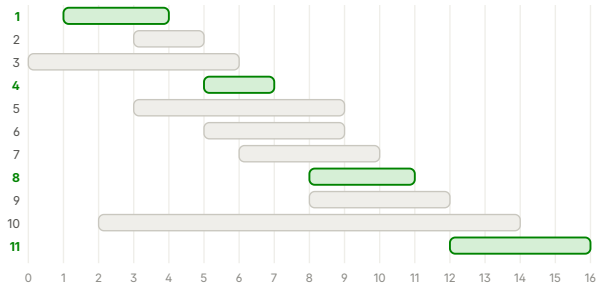
NON SEMPRE FUNZIONA

Con gli intervalli **pesati** la scelta "finisce prima" non è sicura (un intervallo che finisce presto può valere poco) ⇒ serve la PD (§5.4). Con lo zaino **0/1** la scelta "miglior rapporto valore/peso" non è sicura ⇒ PD (§5.8). Con lo zaino **frazionario** sì (§6.4).

6.2 Selezione di intervalli (non pesati)

Problema: dati n intervalli $[s(i), e(i))$, sceglierne il **massimo numero** a due a due compatibili (è il §5.4 con tutti i valori $= 1$).

Th 4 (scelta greedy): sia g l'intervallo che **termina per primo**. Esiste una soluzione ottima che inizia con g. *Dim.* Sia $i_1 i_2 \dots i_k$ ottima. Se $i_1 \neq g$: $e(g) \leq e(i_1) \leq s(i_2)$, quindi $g\, i_2 \dots i_k$ è ancora compatibile e ha la stessa cardinalità k: è ottima e inizia con g. ∎



Intervalli ordinati per tempo di fine: la scelta greedy prende 1, 4, 8, 11 (verde).

SELEZIONE GREEDY — Θ(N LOG N)

```
SelezioneGreedy(s, e, n)  // ordinati per fine
  last ← 0;  e[0] ← -∞
  for i ← 1 to n do
    if s[i] ≥ e[last] then   // compatibile
      print i
      last ← i
```

6.3 Schedulazione con tempo medio di completamento minimo integrazione

Problema: n lavori con durate $t_1, \dots, t_n$, una sola macchina. Trovare l'ordine che minimizza il tempo medio di completamento $\frac{1}{n}\sum_h f(i_h)$, con $f(i_h) = t_{i_1} + \dots + t_{i_h}$.

Scelta greedy: esegui per primo il lavoro **più corto** (e così via: ordina per durata crescente). **Costo** $\Theta(n \log n)$.

PERCHÉ (SCAMBIO)

Se in una soluzione ottima il più corto j^* non è primo, scambialo col primo: i tempi di completamento dei lavori da j^* in poi non cambiano, quelli prima diminuiscono o restano uguali ⇒ la media non peggiora.

Inoltre $c(S) = t_{i_1} + \frac{n-1}{n} c(S')$: il primo lavoro "pesa" su tutti, per questo deve essere il più corto.

6.4 Zaino frazionario integrazione

Problema: come lo zaino, ma posso prendere una **frazione** $q_i \le w_i$ di ogni oggetto; il valore è $\sum q_i \frac{v_i}{w_i}$.

Scelta greedy: l'oggetto con il massimo **valore specifico** $\frac{v_i}{w_i}$, nella quantità $\min(w_i, W)$.

ALGORITMO — Θ(N LOG N)

```
ZainoFrazionario(w, v, n, W)
  ordina per v[i]/w[i] decrescente
  i ← 1;  S ← ∅
  while i ≤ n and w[i] ≤ W do
    S ← S ∪ {(i, w[i])}      // intero
    W ← W - w[i];  i ← i + 1
  if i ≤ n and W > 0 then
    S ← S ∪ {(i, W)}         // una frazione
  return S
```

CONTROESEMPIO PER LO ZAINO 0/1

$W = 4$; pesi $w = (3, 2, 2)$, valori $v = (5, 3, 3)$. Rapporti: $\frac{5}{3} > \frac{3}{2} = \frac{3}{2}$.

- Frazionario (greedy, ottimo): oggetto 1 intero + metà del 2 ⇒ $5 + 1.5 = 6.5$.
- 0/1 con la stessa idea: solo l'oggetto 1 ⇒ valore 5.
- Ottimo 0/1: oggetti 2 e 3 ⇒ valore **6**.

Nello 0/1 la scelta greedy non è sicura: serve la PD.

TEORIA COLLEGATA

7 Code di priorità e Dijkstra

Lo heap non serve solo per ordinare: è la struttura che rende veloce Dijkstra. Utile per le domande di teoria e per motivare le complessità.

7.1 Rilassamento e idea di Dijkstra

Bellman-Ford e Dijkstra mantengono per ogni vertice una stima $d[v] \geq \delta(s, v)$ e un predecessore $p[v]$. L'operazione base è il **rilassamento**: se passando per u arrivo a v spendendo meno, aggiorno.

Dijkstra (pesi **non negativi**) estrae ogni volta il vertice non ancora fissato con d minima: la sua distanza è definitiva, e ne rilassa gli archi uscenti. Serve una struttura che estragga il minimo in fretta: la **coda di priorità**.

RILASSAMENTO

```
relax(u, v, w)
  if d[v] > d[u] + w(u,v) then
    d[v] ← d[u] + w(u,v)
    p[v] ← u
```

7.2 Coda di priorità con un min-heap

Operazioni: `Accoda` (inserisce con priorità), `Testa` (elemento di priorità minima), `Decoda` (lo rimuove), `DecreaseKey` / `IncreaseKey` (cambiano la priorità).

Implementazione: un **min-heap** (ogni padre $\leq$ figli) di coppie (chiave, priorità). Per fare DecreaseKey bisogna sapere **dove** sta una chiave nello heap: si affianca una struttura di ricerca $C.T$ (albero bilanciato o array indicizzato dalle chiavi) che memorizza la posizione, aggiornata da ogni `Swap`.

- **Decrease**: abbasso la priorità e faccio **risalire** l'elemento verso la radice.
- **Increase**: la alzo e faccio **scendere** con Heapify.
- **Decoda**: scambio radice e ultimo, accorcio, Heapify(1).

Tutte $O(\log n)$; `Testa` $O(1)$.

OPERAZIONI PRINCIPALI

```
Decrease(C, p, v)        // risale
  if C.A[p].v > v then
    C.A[p].v ← v
    x ← parent(p)
    while x > 0 and C.A[x].v > v do
      Swap(C, x, p)      // aggiorna le pos.
      p ← x;  x ← parent(p)

Accoda(C, d, v)
  C.size ← C.size + 1
  Insert(C.T, d, C.size)
  C.A[C.size] ← (d, +∞)
  Decrease(C, C.size, v)

Decoda(C)
  x ← C.A[1].k
  Swap(C, 1, C.size)
  C.size ← C.size - 1
  Delete(C.T, x);  Heapify(C, 1)
```

7.3 Dijkstra e la sua complessità

DIJKSTRA CON CODA DI PRIORITÀ

```
Dijkstra(G, s)
  Q ← coda con tutti i v a priorità ∞
  DecreaseKey(Q, s, 0)
  while Q.size > 0 do
    u ← Testa(Q)
    d[u] ← priorità di u    // definitiva
    Decoda(Q)
    for each v ∈ Adj[u] do
      if v è ancora in Q and
         d[u] + w(u,v) < priorità di v then
        DecreaseKey(Q, v, d[u] + w(u,v))
        p[v] ← u
```

Implementazione	Estrai min	Rilassa	Totale
min-heap	$O(\log \lvert V\rvert)$	$O(\log \lvert V\rvert)$	$O((\lvert V\rvert + \lvert E\rvert)\log \lvert V\rvert) = O(\lvert E\rvert \log \lvert V\rvert)$
array non ordinato	$O(\lvert V\rvert)$	$O(1)$	$O(\lvert V\rvert^2)$

QUALE CONVIENE?

Lo heap conviene se $|E| \log |V| < |V|^2$, cioè per grafi **non troppo densi** ($|E| < |V|^2/\log |V|$). Per grafi densi ($|E| \approx |V|^2$) è meglio l'array.

PESI NEGATIVI

Dijkstra richiede pesi ≥ 0. Con pesi negativi (come nel grafo del 16/06) si usa Bellman-Ford / la PD del §5.9.

DA RIPASSARE LA SERA PRIMA

8 Formulario e checklist finali

Tutto quello che va ricordato, in due pagine.

8.1 Tutte le complessità

Algoritmo / problema	Tempo
MaxSum: ingenuo / incrementale / Kadane	$\Theta(n^3)$ / $\Theta(n^2)$ / $\Theta(n)$
Insertion Sort: migliore / medio / peggiore	$\Theta(n)$ / $\Theta(n^2)$ / $\Theta(n^2)$
Selection Sort (sempre)	$\Theta(n^2)$
Merge Sort (sempre)	$\Theta(n \log n)$
Heapify / BuildHeap	$O(\log n)$ / $\Theta(n)$
Heap Sort	$\Theta(n \log n)$
Partiziona	$\Theta(n)$
Quick Sort: peggiore / medio	$\Theta(n^2)$ / $\Theta(n \log n)$
Limite inferiore per confronti: peggiore e medio / migliore	$\Omega(n \log n)$ / $\Omega(n)$
Dijkstra: heap / array	$O(\|E\| \log \|V\|)$ / $O(\|V\|^2)$

Programmazione dinamica / greedy	Tempo
Fibonacci: ricorsivo / PD	$\Theta(\varphi^n)$ / $\Theta(n)$
Taglio del tronco: ricorsivo / PD	2^L / $\Theta(L^2)$
Intervalli pesati (con ordinamento)	$\Theta(n \log n)$
LCS	$\Theta(nm)$
ABR ottimo	$\Theta(n^3)$
Zaino 0/1 (pseudopolinomiale)	$\Theta(nK)$
Bellman-Ford (PD)	$\Theta(\|V\| \cdot \|E\|)$
Tutte le coppie: espansione / raddoppio / F-W	$\Theta(n^4)$ / $\Theta(n^3 \log n)$ / $\Theta(n^3)$
Cammino semplice max: generale / DAG	NP-completo / $\Theta(\|V\| + \|E\|)$
Greedy: intervalli, SPT, zaino frazionario	$\Theta(n \log n)$

8.2 Formule da avere in mano

SERIE

$\sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1}$ $(r \neq 1)$

$\sum_{i \geq 0} r^i = \frac{1}{1-r}$ $(r < 1)$

$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

$\sum_{i \geq 0} \frac{i}{2^i} = 2$

LOGARITMI

$\log x^k = k \log x$

$\log_a b = \frac{\log_c b}{\log_c a}$

$a^{\log_b c} = c^{\log_b a}$

$\log n! = \Theta(n \log n)$

ALTEZZE

$n/b^i \leq 1 \Rightarrow h = \log_b n$

$(\frac{c}{d})^i n \leq 1 \Rightarrow h = \log_{d/c} n$

$n^{1/b^i} \leq c \Rightarrow h \approx \log_b \log n$

$n - i \leq 1 \Rightarrow h \approx n$

8.3 Le quattro checklist

1 ORDINAMENTO

- Pseudocodice completo di **tutte** le funzioni (anche ausiliarie).
- Base degli indici (0 o 1?) e n.
- Iterazione k ↔ valore dell'indice del ciclo più esterno.
- Una riga di motivazione per configurazione.
- Gli elementi finali sono gli stessi di quelli iniziali.

2 RICORRENZE

- Albero con 2–3 livelli e molteplicità sugli archi.
- Tabella: livello · costo locale · costo totale; termine generale.
- Ragione r e serie fino a k.
- h (o h_{min}, h_{max}) dal caso base.
- Sostituzione e semplificazione: Θ oppure $\Omega \leq T \leq O$.

3 RICORSIVO → ITERATIVO

- Tabella delle variabili: prima / dopo / modificata ⇒ stack.
- **last**: parametro sempre diverso tra le due chiamate.
- `while call or S ≠ ∅`; Fase 1 / risalita con Top.
- Ogni "return": **ret**, **last**, Pop (o **call ← false**).
- Funzioni esterne nello stesso ordine.

4 PROGRAMMAZIONE DINAMICA

- Soluzione ammissibile e costo, in formule.
- Scelta + sottoproblema; numero di sottoproblemi.
- Th 1, Th 2, Th 3 (per assurdo) in breve.
- Ricorrenza con casi base; ordine di riempimento.
- Algoritmo + ricostruzione; tempo e spazio.
- Simulazione: tabella completa, risultato, soluzione.

★ STRATEGIA IL GIORNO DELL'ESAME

Leggi tutti e 4 gli esercizi (2 minuti). Parti da quello che sai fare meglio, poi gli altri. Gli esercizi 1 e 4 hanno parti "di memoria" (pseudocodice, ricorrenze): scrivile subito, poi passa alle simulazioni. Lascia 10 minuti per ricontrollare i conti delle simulazioni: è lì che si perdono più punti.